Analisi statica e verifica del software

Giuseppe Di Blasi
Università di Parma
giuseppe.diblasi@studenti.unipr.it

A.A. 2026/2027

ABSTRACT

Appunti sul corso di Analisi statica e verifica del software

Indice

Indice

1 Lezione 1 del 23 settembre 2026 4
1.1 Introduzione al corso 4
1.2 Affidabilità del software 4
1.3 Validazione vs Verifica 5
1.4 Verifica del software 5
2 Lezione 2 del 24 settembre 2026 8
2.1 Teoria dei reticoli 8
2.2 Ordinamenti parziali (PoSet) 8
2.3 Diagrammi di Hasse 9
2.4 Upper bound e Lower bound 11
2.5 Lattice (Reticoli) 14
3 Lezione 3 del 25 settembre 2026 16
3.1 Relazioni 17
3.2 Catene (Chain) 20
4 Lezione 4 del 30 settembre 2026 23
4.1 Punti fissi (Fixed points) 23
5 Lezione 5 del 1 ottobre 2026 26
5.1 Imp, linguaggio toy 26
6 Lezione 6 del 2 ottobre 2026 28
7 Lezione 7 del 7 ottobre 2026 31
7.1 Collective semantics 31
7.2 Analisi Data Flow (Data Flow Analysis, DFA); Slide 04 32
8 Lezione 8 dell'8 ottobre 2026 34
8.1 Available expressions, pt. 2 34
8.2 Altre analisi Dataflow 37
8.3 Lattice in analisi dataflow 38
8.4 Analisi distributive 39
8.5 Propagazione di costanti 39
9 Lezione 9 del 9 ottobre 2026 41
9.1 Approssimazioni 41
9.2 Conclusioni 46
10 Lezione 10 del 14 ottobre 2026 48
10.1 Dominio concreto 48
10.2 Dominio astratto 48
10.3 Connessioni di Galois 52
11 Lezione 11 del 15 ottobre 2026 54
11.1 Inserzioni di Galois 54
11.2 Semantica concreta 59
12 Lezione 12 del 16 ottobre 2026 61
12.1 Semantica astratta 61
12.2 Soundness di un'astrazione 66

1 Lezione 1 del 23 settembre 2026

1.1 Introduzione al corso

Il corso di Analisi statica e verifica del software ha come obiettivo quello di fornire tecniche e metodologie per costruire programmi di analisi statica per poter verificare la correttezza e la validità di un programma.

1.2 Affidabilità del software

Cosa si intende per **affidabilità**?

Una definizione secondo l'IEEE è: *Abilità del sistema, o di un componente, di rispettare i requisiti sotto determinate condizioni.*

L'affidabilità di un sistema software è composta da 3 diverse attività:

- Error prevention
- Fault detection
- Rimozione e correzione di bug

Perché l'affidabilità del software è importante?

La correzione di bug è parte integrante del ciclo di sviluppo di un sistema software, e una eventuale correzione dopo che il sistema è stato rilasciato comporta un costo maggiore rispetto a correggerlo prima della messa sul mercato.

Per esempio gli Stati Uniti perdono 2,116 mila miliardi di dollari del loro PIL a causa di bug, falle di sicurezza in software scadenti.

Quindi la presenza di bug non solo comporta problemi d'esecuzione per il programma ma anche costi economici e temporali.

1.2.1 Esempi di danni economici

1.2.1.1 Ariane 5 Flight 501 (1996)

Era un test di un razzo spaziale, si è schiantato appena dopo il decollo per un bug all'interno del codice dovuto ad un cast da 64 bit float a 16 bit unsigned. Sono stati persi 370 milioni di dollari per quell'incidente.

1.2.1.2 Pentium FDIV bug (1994)

Errore nel processore Intel per le divisioni tra float, ha costretto Intel a rimuovere il processore dal mercato e correggere l'errore. Anche in questo caso il costo economico è piuttosto elevato di circa 475 milioni di dollari.

1.2.1.3 CrowdStrike (2024)

Questo errore nel software ha portato alla cancellazione di 5 mila voli. Impattando anche sui sistemi informatici di banche, enti governativi e strutture sanitarie, portando a un costo totale di 5 miliardi di dollari.

L'errore era dovuto alla dereferenziazione di un puntatore null.

1.2.2 Software immutabile

Nel caso della blockchain Etherium era stato scoperto un bug che comportava la perdita del portafoglio per via dell'ordine d'esecuzione di due istruzioni che dovevano essere invertite. La blockchain può essere vista come un software immutabile, il che comporta che anche i bug siano immutabili.

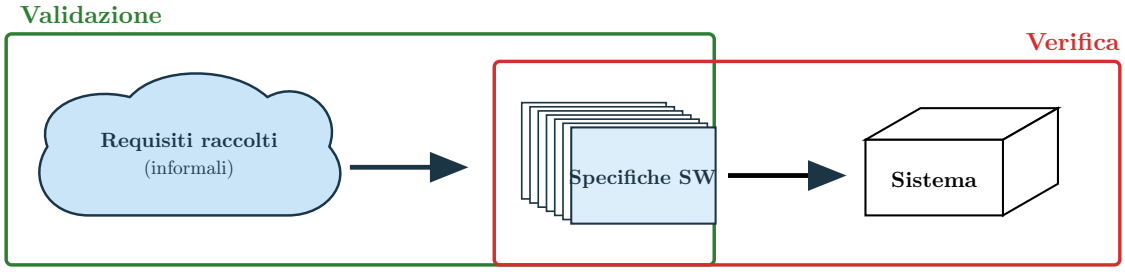
Questo ha portato alla creazione di un fork di Etherium per correggere l'errore.

1.3 Validazione vs Verifica

Cosa si intende per validazione del software? Risponde alla domanda: *Il software rispecchia i bisogni dell'utente?*

Quindi ci chiediamo se stiamo costruendo il software in base alle richieste dell'utente.

Mentre la verifica risponde alla domanda: *Il software soddisfa le specifiche e i requisiti?*



- Validazione: usability testing, feedback da utente e stesura dei requisiti
- Verifica: testing, analisi statica

Ci sono casi in cui alcune specifiche sono validabili, ma non verificabili e viceversa. Esempio di specifica non verificabile:

Se l'utente preme il bottone per chiamare l'ascensore a un determinato piano dell'edificio l'ascensore deve arrivare entro un determinato tempo al piano desiderato

Esempio di specifica verificabile:

Se l'utente preme il bottone per chiamare l'ascensore a un determinato piano dell'edificio l'ascensore deve arrivare entro 30 secondi al piano desiderato

1.4 Verifica del software

A cosa serve la verifica del software?

Serve a verificare la correttezza di un software rispetto:

- la **correttezza funzionale**, il programma offre i servizi definiti nei requisiti e specifiche

- l'**assenza di errori non-funzionali**, ovvero:
  - rispetto dei vincoli spaziali
  - rispetto dei vincoli temporali
  - soddisfare vincoli di sicurezza

In questo corso per verifica del software si intende ricercare certificati di correttezza.

Purtroppo non esiste un tool di analisi statica che sia:
- **Automatico**: senza interazione umana
- **Potente**: capacità di trovare proprietà non banali, ovvero proprietà semantiche
- **Sound**: Trovare errori evitando falsi negativi.
- **Completo**: Trovare errori e garantire la correttezza se il programma è corretto, evita falsi positivi.

Un tool con tutte queste proprietà non esiste.

Teorema 1 — Teorema di Rice (Rice's Theorem)

Nella teoria della calcolabilità, il **teorema di Rice** dice che tutte le proprietà **non-banali** di un programma in un linguaggio Turing-completo sono **indecidibili**.

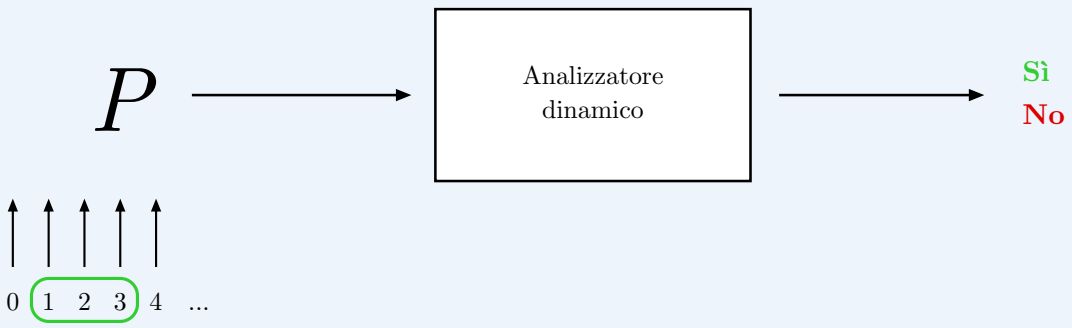
Per proprietà semantica si intende il comportamento del programma, per esempio se l'esecuzione termina per tutti i valori presenti nell'insieme di input.

Siccome non possiamo avere un tool perfetto siamo costretti a scendere a compromessi, questo è un po' così in tutta l'informatica ovvero cercare il compromesso migliore per il nostro obiettivo.

Quindi dobbiamo farci la domanda: *A cosa possiamo rinunciare?*

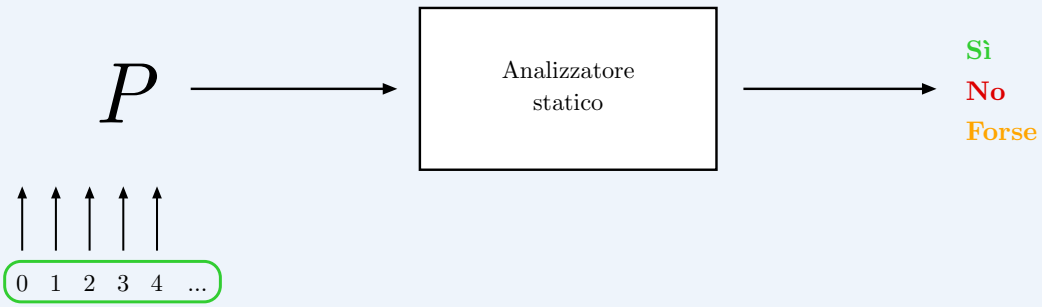
Sicuramente non possiamo rinunciare alla soundness, dato che l'obiettivo del corso è dimostrare la correttezza del software. Sicuramente avere sia completezza che soundness è molto difficile, saranno le due proprietà su cui dovremo fare più compromessi.

Definizione 1 — Analisi dinamica



L'analisi dinamica lavora su un sotto insieme dei possibili valori di input, dunque manca di completezza.

Definizione 2 — Analisi statica



L'analisi statica ci permette di avere completezza su tutti gli input possibili per il programma, ma a discapito di precisione.

Le varie tecniche per l'analisi statica sono:

- Interpretazione astratta
- Model checking
- Symbolic execution

2 Lezione 2 del 24 settembre 2026

2.1 Teoria dei reticoli

Per arrivare a parlare dei reticoli, in questa lezione si inizia con un ripasso dei concetti di insiemistica.

Partiamo col definire un insieme:

Definizione 3 — Insieme

Un insieme S è una collezione di elementi non ripetuti.

2.1.1 Implicazione

$$S_1 \subset S_2 \triangleq \forall\, s \in S_1 \Rightarrow s \in S_2$$

2.1.2 Or logico

$$S_1 \cup S_2 \triangleq \{s \mid s \in S_1 \vee s \in S_2\}$$

2.1.3 And logico

$$S_1 \cap S_2 \triangleq \{s \mid s \in S_1 \wedge s \in S_2\}$$

2.2 Ordinamenti parziali (PoSet)

Definizione 4 — Insieme parzialmente ordinato

Un insieme si dice parzialmente ordinato se rispetta le seguenti proprietà:

- **Riflessiva:** $\forall\, x \in X \Rightarrow x \sqsubseteq x$
- **Anti-simmetrica:** $\forall\, x, y \in X, x \sqsubseteq y \wedge y \sqsubseteq x \Rightarrow x = y$
- **Transitiva:** $\forall\, x, y, z \in X, x \sqsubseteq y \wedge y \sqsubseteq z \Rightarrow x \sqsubseteq z$

L'insieme è ordinato parzialmente perché non tutti gli elementi sono confrontabili tra loro, altrimenti si parlerebbe di ordinamento totale.

Per rappresentare i PoSet usiamo la seguente notazione:

$$< X, \sqsubseteq > \quad \longrightarrow \quad < \mathcal{P}(\mathbb{Z}), \subseteq >$$

- X, è l'insieme degli elementi
- $\sqsubseteq$, è l'operatore di ordinamento

Definizione 5 — Power set (Insieme delle parti)

Dato un insieme S, $\mathcal{P}$(S) è l'insieme di tutti i sottoinsiemi di S.

2.2.0.1 Esempi:

- $\mathcal{P}(\varnothing) = \{\varnothing\}$
- $\mathcal{P}(\mathbb{Z}) = \{\varnothing, \{0\}, \{1\}, \{0,1\}, ...\}$
- $\mathcal{P}$(S), anche S è un insieme che fa parte dell'insieme delle parti.

Esempio 1 — Verificare un Poset

$\mathcal{P}$(S) è un PoSet? Per decidere se è un PoSet dobbiamo dimostrare le 3 proprietà:

- **Riflessivo:** $\forall X \in \mathcal{P}(S) \quad \Rightarrow \quad X \subseteq X$
- **Anti-simmetrico**: $\forall X, Y \in \mathcal{P}(S), \quad X \subseteq Y \wedge Y \subseteq X \Rightarrow X = Y$
- **Transitiva:** $\forall X, Y, Z \in \mathcal{P}(S), \quad X \subseteq Y \wedge Y \subseteq Z \Rightarrow X \subseteq Z$

Nota

Se $< \mathcal{P}(\mathrm{S}), \sqsubseteq >$ è un PoSet, è uguale a: $< \mathcal{P}(\mathrm{S}), \sqsupseteq >$?

In linea generale sì, questa proprietà è valida.

$< \mathbb{N}, \leq >$ è un PoSet.

$< \mathbb{N}, \geq >$ è un PoSet.

$< \mathbb{N}, < >$ non è un PoSet.

2.3 Diagrammi di Hasse

È un modo per poter rappresentare graficamente un PoSet.

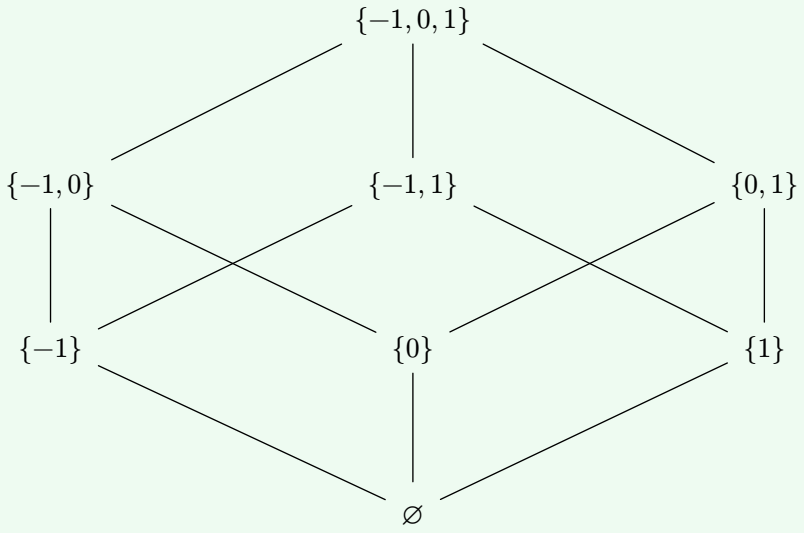
Dando una definizione formale:

Definizione 6 — Diagramma di Hasse

Dato $< \mathrm{X}, \sqsubseteq >$, traccio una linea per collegare due elementi **SOLO SE:**

- $\mathrm{x} \sqsubset \mathrm{y}$
- $\nexists\, \mathrm{z} \in \mathrm{X}, \mathrm{x} \sqsubset \mathrm{z} \sqsubset \mathrm{y}$

Esempio 2 — Diagramma di Hasse



Vediamo alcuni esempi di PoSet e di diagrammi di Hasse:

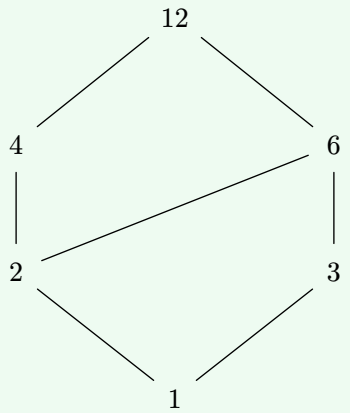
Esempio 3

Prendiamo l'insieme dei divisori di 12, $X = \{1, 2, 3, 4, 6, 12\}$ e operatore di ordinamento $x \sqsubseteq y$ se x divide y. Quindi definendolo formalmente abbiamo: $< X, | >$

Verifichiamo che siano rispettate le 3 proprietà:

- **Riflessiva**: $\forall x \in X, x \sqsubseteq x$, in questo caso ogni numero divide se stesso.
- **Anti-simmetrica**: $\forall x, y \in X, \text{se } x \sqsubseteq y \wedge y \sqsubseteq x, \text{ quindi: } x \mid y \wedge y \mid x \Rightarrow x = y$. Quindi se $x \mid y, x \leq y$, se $y \mid x, y \leq x$, quindi $x = y$
- **Transitiva**: $\forall x, y, z \in X, \text{se } x \sqsubseteq y \wedge y \sqsubseteq z \Rightarrow x \sqsubseteq z$, presi 3 numeri, se x divide y, e y divide z, allora x divide z. Anche questa proprietà è rispettata.

Possiamo rappresentare questo PoSet come un diagramma di Hasse:



2.4 Upper bound e Lower bound

Quando lavoriamo con diagrammi di Hesse è fondamentale, per i nostri scopi riguardanti l'analisi statica, quella di trovare gli upper bound e i lower bound da un dato insieme S all'interno di un diagramma.

Definiamo ora upper bound e il least upper bound.

Definizione 7 — Upper bound e least upper bound (lub)

Dato un PoSet $< X, \sqsubseteq >, S \subseteq X$

- $s \in X$ è un **upper bound** se: $\forall s' \in S, \quad s \sqsupseteq s'$
- $s \in X$ è un **least upper bound** (lub) se: $\forall s' \in UB, s \sqsubseteq s'$

Diamo un'occhiata ora alle definizioni di lower bound e greatest lower bound.

Definizione 8 — Lower bound e greatest lower bound (glb)

Dato un PoSet $< X, \sqsubseteq >, S \subseteq X$

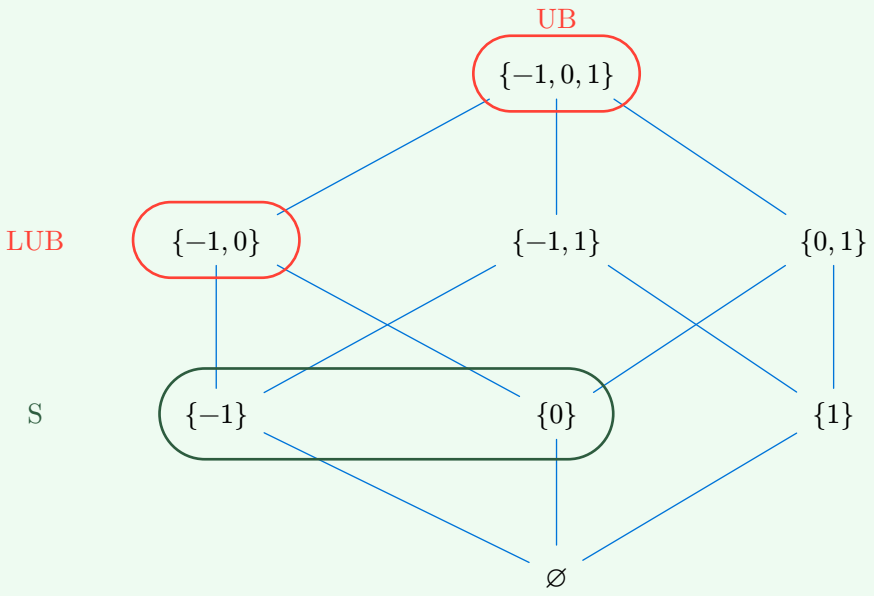
- $s \in X$ è un **lower bound** di S se $\forall s' \in S, s \sqsubseteq s'$
- $s \in X$ è un **greatest lower bound** se $\forall s' \in LB, s' \sqsubseteq s$

Nota

Il **lub** e **glb** non sempre esistono, ma nel caso in cui dovessero esistere, sono **unici**.

Esempio 4 — Upper bound e least upper bound di un diagramma di Hasse

Vediamo ora un esempio di upper bound e least upper bound.



Nel diagramma qui sopra possiamo vedere che i due insiemi

$$\{-1, 0, 1\}, \{-1, 0\}$$

sono entrambi upper bound per l'insieme $S = \{\{-1\}, \{0\}\}$. Il **least upper bound** in questo caso è l'insieme $\{-1, 0\}$.

Seguendo la struttura del diagramma, in questo caso il LUB è sempre il primo UB che troviamo muovendoci verso l'alto lungo il diagramma.

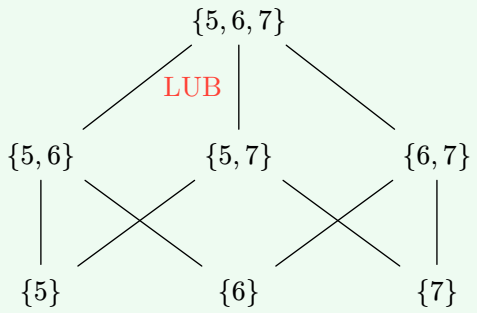
Esempio 5 — Esempio con pseudocodice

Vediamo un caso in cui un diagramma di Hasse può tornare utile per i nostri scopi.

Prendiamo in considerazione il seguente pseudocodice:

```
1 int arr[10]
2 if (function()) then
3   | x ← 5
4 else
5   | x ← 7
6 end
7 arr[x]
```

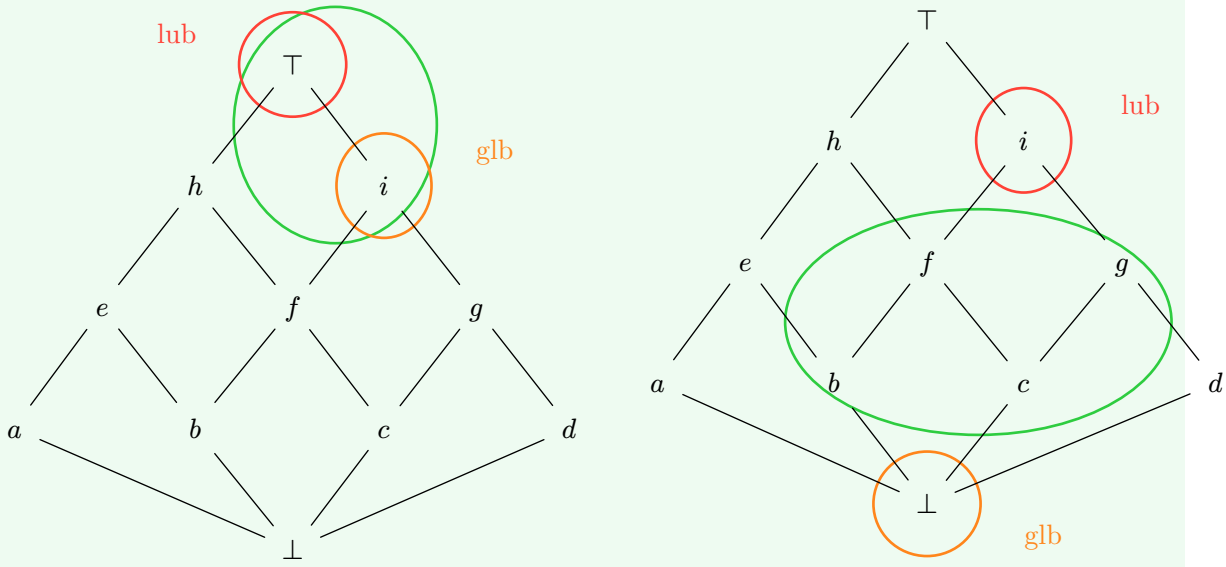
Per questo pseudocodice possiamo creare un diagramma di Hasse con i possibili valori che x può assumere:



In questo caso il LUB è l'insieme $\{5, 7\}$, questo ci permette di controllare se x avrà un valore compreso tra 0 e 9 quando proviamo ad accedere all'array *arr*. Se avessimo scelto l'insieme $\{5, 7, 12\}$, che fa sempre parte del dominio della variabile x, avremmo ottenuto un falso positivo, dato che x può assumere il valore 12 che è fuori dal bound $\{0..9\}$ dell'array.

Seguono esempi di diagrammi di Hasse con GLB e LUB:

Esempio 6 — Diagrammi di Hesse con lub e glb



Il diagramma di Hasse di sinistra, prendendo come insieme $S = \{\top, \mathrm{i}\}$, abbiamo come glb l'elemento i, questo perché il glb, se esiste, deve essere unico; l'altra scelta sarebbe ricaduta tra f e g che possono essere glb per l'insieme S.

Invece per quello di destra, prendendo come insieme $S = \{\mathrm{f}, \mathrm{g}, \mathrm{b}, \mathrm{c}\}$, abbiamo come glb l'elemento $\bot$, e come lub i. Sempre per lo stesso discorso fatto per il diagramma di sinistra, lub e glb se esistono sono unici; i collega direttamente tutti gli elementi di S, stessa cosa per $\bot$.

Definizione 9 — Supremo/Top/Maximum

Un top di S $\subseteq$ X se: x $\in$ S, e x è lub.

Definizione 10 — Infinimum/Bottom/Minimum

Un bottom di S $\subseteq$ X se x $\in$ S e x è glb.

Definizione 11 — Operatori lub e glb

Possiamo definire anche i due operatori lub e glb come segue:

Per lub usiamo $\sqcup$ S = lub

Per glb usiamo $\sqcap$ S = glb

Valgono anche le seguenti equivalenze:

$$\sqcup \{\mathrm{a}, \mathrm{b}\} \rightarrow \mathrm{a} \sqcup \mathrm{b}$$

$$\sqcap \{\mathrm{b}, \mathrm{c}\} \rightarrow \mathrm{b} \sqcap \mathrm{c}$$

Nota

Quando lavoriamo con PoSet possiamo definire anche gli operatori di lub e glb:

$$< X, \sqsubseteq, \sqcup, \sqcap >$$

2.5 Lattice (Reticoli)

Una cosa importante in interpretazione astratta per l'analisi del software è il concetto di **lattice**, o reticolo. Vedremo più avanti nel corso cosa ci permette di fare il concetto di reticolo per i nostri scopi.

Intanto diamo una definizione di reticolo:

Definizione 12 — Reticolo (Lattice)

Partendo da un PoSet $< X, \sqsubseteq >$ è un reticolo se vengono rispettate le seguenti proprietà:

- Join semi-lattice: $\forall\ x, y \in X, \quad \exists\, x \sqcup y$; *Per ogni coppia* x *e* y *deve sempre esistere il lub.*
- Meet semi-lattice: $\forall\ x, y \in X, \quad \exists\, x \sqcap y$; *Per ogni coppia* x *e* y *deve sempre esistere il glb.*

Il termine reticolo e lattice indicano la stessa cosa.

Esempio 7 — Esempi di join, meet semi-lattice e di reticolo

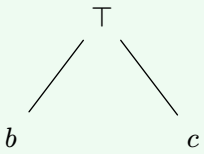


Figura 1. È un join semi-lattice

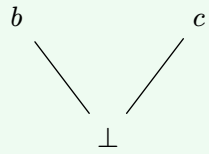


Figura 2. È un meet semi-lattice

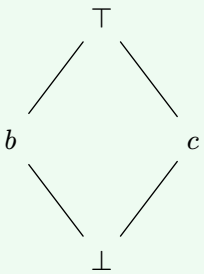


Figura 3. Questo è un reticolo

Una domanda sollevata dal prof, se prendessimo il PoSet: $< \mathcal{P}(X), \subseteq, \cup, \cap >$ è un reticolo. Ma se X è infinito, allora il reticolo è infinito anche in altezza?

Qualsiasi sottoinsieme di X, ottenuto tramite intersezione o unione, rimane sempre nell'insieme delle parti di X.

Per esempio: $< \mathbb{Z}, \leq, \max, \min >$ è un reticolo.

Definizione 13 — Lub e Glb di un insieme S

Possiamo dare un'altra definizione di *lub* e *glb* per un insieme S:

$$\text{lub: } \sqcup S = \min\{s' \mid \forall\, s \in S, s' \sqsupseteq s\}$$

$$\text{glb: } \sqcap S = \max\{s' \mid \forall\, s \in S, s' \sqsubset s\}$$

Nota

Un altro esempio è quello dei linguaggi regolari, prendiamo in considerazione:

$< L_{reg}, \subseteq, \cup, \cap >$, questo è un PoSet.

Questo perché l'intersezione e l'unione di due o più linguaggi regolari è sempre un linguaggio regolare. Ma nel caso in cui dovessi prendere in considerazione un'unione infinita come:

$$L_i = \{a^n, b^n \mid n \in \mathbb{N}\}$$

$$\cup L_i = \{a^n, b^n \mid n \in \mathbb{N}\}$$

Con questa unione infinita non abbiamo più un linguaggio regolare, è un reticolo, ma non completo.

La definizione di reticolo completo sarà nella prossima lezione.

3 Lezione 3 del 25 settembre 2026

Nella lezione precedente abbiamo parlato di operazioni insiemistiche, insiemi parzialmente ordinati (PoSet), insieme delle parti (Power Set), diagrammi di Hasse, operatori lub e glb e una introduzione ai reticoli.

In questa lezione introduciamo il concetto di lattice (reticolo) completo. Prendiamo in considerazione il seguente PoSet:

$< \mathcal{P}(\mathrm{X}), \subseteq, \cup, \cap >$

- Come elemento top abbiamo: X
- Come bottom: $\varnothing$

Se prendiamo il PoSet:

$< \mathbb{Z}, \leq, \max, \min >$ è un PoSet e anche un reticolo, ma non è **completo**.

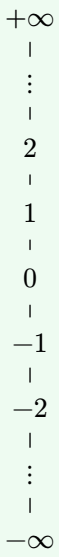
Definizione 14 — Complete lattice (Reticolo completo)

Dato: $< \mathrm{X}, \sqsubseteq, \sqcup, \sqcap >$ è completo se:

- Qualunque sottoinsieme di X ha **lub**
- Deve avere il bottom element

Esempio 8 — Esempio reticolo completo

Se prendiamo $< \mathbb{Z} \cup \{-\infty, +\infty\}, \leq, \max, \min >$ questo è un reticolo completo. Qui abbiamo aggiunto $-\infty$ e $+\infty$ all'insieme.



Se $< \mathrm{X}, \sqsubseteq, \sqcup, \sqcap >$ è completo se ha $\top$ (Top) e $\bot$ (Bottom).

Possiamo riscriverlo anche così, specificando gli elementi top e bottom:

$< \mathrm{X}, \sqsubseteq, \sqcup, \sqcap, \top, \bot >$

Nota

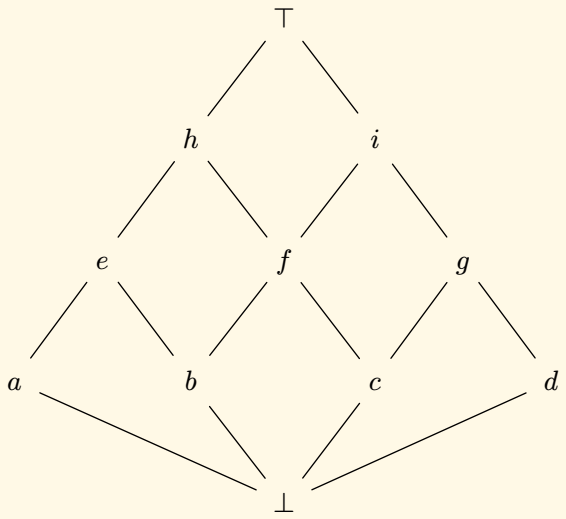
Se abbiamo solo l'operatore **lub**, o anche il **glb**, possiamo dire che partendo dalla definizione del **lub**, possiamo indurre il **glb** e viceversa. **Questo vale solamente per reticoli completi.** I due concetti, si dice che sono in dualità; stessa cosa $\top$ e $\bot$.

$\sqcup$ induce $\sqcap \rightarrow \quad \sqcup S = \sqcup \{y \mid \forall\, s \in S, y \sqsubseteq x\}$

$\sqcap$ induce $\sqcup \rightarrow \quad \sqcap S = \sqcap \{y \mid \forall\, s \in S, x \sqsubseteq y\}$

Se prendiamo $< \mathbb{Z}, \cup \{-\infty, +\infty\}, \leq, \max, \min >$ questo è un reticolo completo.

Prendendo come esempio un vecchio reticolo possiamo calcolare lub e glb:



$\text{lub} \rightarrow \sqcup \{\bot, c\} = c$

$\text{glb} \rightarrow \sqcap \{i, \top\} = i$

3.1 Relazioni

Iniziamo ora a dare un'occhiata alle relazioni, partiamo definendo formalmente cos'è una relazione:

Definizione 15 — Relazione

Dati due insiemi X, Y , il loro prodotto cartesiano è definito come:

$$X \times Y = \{(x, y) \mid x \in X, y \in Y\}$$

.

Una relazione R è definita:

$R \subseteq X \times Y$

$(x, y) \in R$, x r y -> relazione tra x e y.

L'operatore **r**, indica x in relazione con y.

Gli ordinamenti parziali, ovvero i PoSet che abbiamo visto fino ad ora, sono un tipo di relazione. Altri tipi di relazione sono le funzioni:

Definizione 16 — Funzioni

$\forall (x, y) \in R, \nexists (x', y'), x = x' \land y \neq y'$

$f \subseteq X \times Y \approx f : X \to Y$

- X è il dominio di f.
- Y è il codominio di f.

$(x, y) \in f \approx f(x) = y$

Le relazioni ci permettono di modellare la semantica di un programma come una funzione calcolata su un complete lattice. Per esempio:

Definizione 17 — Funzioni tra PoSet: monotonia, order-embedding e isomorfismo

Prendiamo 2 PoSet e una funzione f che li mette in relazione:

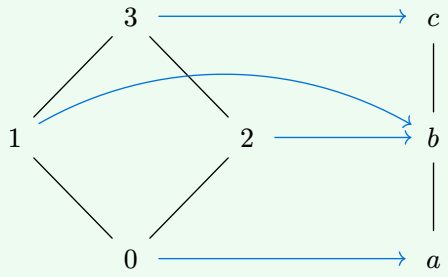
$< X, \sqsubseteq_X > \quad < Y, \sqsubseteq_Y > \quad f : X \to Y$

Possiamo dire che:

- f è **monotona** se: $x_1 \sqsubseteq_X x_2 \Rightarrow f(x_1) \sqsubseteq_Y f(x_2)$
- f è **order-embedding** se: $x_1 \sqsubseteq_X x_2 \Leftrightarrow f(x_1) \sqsubseteq_Y f(x_2)$
- f è un **isomorfismo** se:
  - è un **order-embedding**
  - e se è **suriettiva**: $\forall y \in Y, \exists x \Rightarrow f(x) = y$

Ricordiamo che la funzione è un tipo di relazione.

Esempio 9 — Relazioni e reticoli



- f **monotona**: ✓
- f **order-embedding**: ✗
- f **isomorfismo**: ✗

Prendiamo in considerazione il reticolo qui sopra e vediamo nel dettaglio come dimostrare se f è monotona: riprendendo la definizione dobbiamo comparare ogni possibile coppia:

Coppia in X	**Vale $x_1 \underset{X}{\subseteq} x_2$?**	$f(x_1) \underset{Y}{\subseteq} f(x_2)$**?**
$0 \subseteq 1$	vero	$a \subseteq b \to$ vero

Coppia in X	**Vale $x_1 \underset{X}{\subseteq} x_2$?**	$f(x_1) \underset{Y}{\subseteq} f(x_2)$**?**
$0 \subseteq 2$	vero	$a \subseteq b \rightarrow$ vero
$1 \subseteq 3$	vero	$b \subseteq c \rightarrow$ vero
$2 \subseteq 3$	vero	$b \subseteq c \rightarrow$ vero
$0 \subseteq 3$	transitiva	$a \subseteq c \rightarrow$ vero

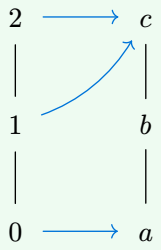
Per quanto riguarda la proprietà dell'order-embedding che non viene rispettata è dovuta al fatto che gli elementi 1 e 2, che non sono comparabili in X vengono mappati sullo stesso valore in Y.

Abbiamo che: $f(1) \underset{Y}{\subseteq} f(2)$ è vera come affermazione, ma non implica che $1 \underset{X}{\subseteq} 2$.

In questo caso non possiamo parlare di isomorfismo perché:

- Non è order-embedding.
- È suriettiva, tutte le $y \in Y$ sono mappate a una $x \in X$.

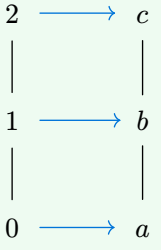
Entrambe le condizioni **DEVONO** essere rispettate.



- f **monotona**: ✓
- f **order-embedding**: ✗
- f **isomorfismo**: ✗

Il caso riportato qui sopra abbiamo f che è monotona, ma non è order-embedding. Guardando più nel dettaglio possiamo vedere che la coppia $(1, 2)$ è confrontabile e abbiamo che: $1 \sqsubseteq 2 \Rightarrow f(1) \sqsubseteq f(2)$

Questa condizione è rispettata, ma siccome dobbiamo prendere in considerazione **TUTTE** le coppie di (x_1, x_2) dobbiamo controllare anche la coppia $(2, 1)$ e abbiamo: $f(2) \sqsubseteq f(1) \nRightarrow 2 \sqsubseteq 1$



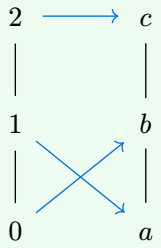
- f **monotona**: ✓
- f **order-embedding**: ✓
- f **isomorfismo**: ✓

Diamo un'occhiata a quest'ultimo esempio, possiamo vedere che f è monotona. Per quanto riguarda l'order embedding in questo caso è rispettato, nello specifico, se dovessimo prendere la coppia di valori $(1, 2)$ e $(2, 1)$ possiamo dire che:

- $f(1) \sqsubseteq f(2) \Rightarrow 1 \sqsubseteq 2$ è vera.

- $f(2) \sqsubseteq f(1) \Rightarrow 2 \sqsubseteq 1$ è vera, dato che entrambe le condizioni sono false.

Controlliamo se questa relazione è isomorfa:
- È order-embedding ✓
- È suriettiva, tutte le $y \in Y$ sono mappate ad almeno una $x \in X$.



- f **monotona**: ✗
- f **order-embedding**: ✗
- f **isomorfismo**: ✗

Verifichiamo che f sia monotona, partiamo dalla definizione di monotona:

$$f \text{ monotona se: } x \sqsubseteq_X y \Rightarrow f(x) \sqsubseteq_Y f(y)$$

Prendiamo $x = 0$ e $y = 1$, e calcoliamo $f(0) = b$ e $f(1) = a$, e otteniamo:

$$0 \sqsubseteq_X 1 \nRightarrow b \sqsubseteq_Y a$$

Come possiamo vedere per i valori appartenenti ad X l'ordinamento viene rispettato, mentre per i valori in Y non viene rispettato, di conseguenza la funzione f non è monotona. Detto volgarmente, guardando il diagramma della relazione si può vedere un incrocio dall'insieme X all'insieme Y.

Definizione 18 — Join/meet preserving function

Dati due PoSet: $< X, \sqsubseteq_X, \sqcup_X, \sqcap_X >$, $< Y, \sqsubseteq_Y, \sqcup_Y, \sqcap_Y >$

Prendiamo in considerazione anche la funzione: $f : X \to Y$

Possiamo definire il **join preserving function**: $f(x_1 \sqcup_X x_2) = f(x_1) \sqcup_Y f(x_2)$

E definire il **meet preserving function**: $f(x_1 \sqcap_X x_2) = f(x_1) \sqcap_Y f(x_2)$

3.2 Catene (Chain)

Il concetto di **catena** ci torna molto utile per il calcolo del punto fisso per determinate funzioni.

Diamo una definizione di catena.

Definizione 19 — Catena

Prendiamo il PoSet : $< X, \sqsubseteq_X >$; prendiamo un sotto insieme $S \subseteq X$, S è una catena se: $\forall x, y \in S$, $x \sqsubseteq y \vee y \sqsubseteq x$, ($S$ è totalmente ordinato.)

Una **catena ascendente** è una sequenza $(\ell_n), n \in \mathbb{N}, \quad i \leq j \Rightarrow \ell_i \sqsubseteq \ell_j$

Diamo ora un'occhiata a un concetto fondamentale per interpretazione astratta.

Definizione 20 — Ascending Chain Condition (ACC)

Prendiamo il PoSet $< X, \sqsubseteq_X >$ soddisfa **ACC** se non esistono catene ascendenti infinite strettamente crescenti. Ogni catena dopo un numero finito di passi si stabilizza; qui torna il concetto di calcolo del punto fisso visto a Linguaggi Interpreti e Compilatori.

$\exists k \geq 0, \forall j \geq k \quad \ell_k = \ell_j$

- Se X è finito -> allora è **ACC**
- $\mathcal{P}(X)$ -> solo se X è finito, allora è **ACC**

Diamo ora un'occhiata a vari esempi di catene che rispettano, e non rispettano la condizione ACC.

Esempio 10 — Esempi di ACC

$< \mathbb{Z}, \leq >$ non è ACC: $0 < 1 < 2 < ...$ è strettamente crescente all'infinito.

$< \mathbb{N}, \leq >$ non è ACC: $0 < 1 < 2 < ...$ è strettamente crescente all'infinito.

$< \mathbb{N}, \geq >$ è ACC: una catena ascendente è $x_0 \geq x_1 \geq x_2 \geq ...$, e non può decrescere all'infinito in $\mathbb{N}$, quindi si stabilizza (es. $5 \geq 3 \geq 1 \geq 0 \geq 0 \geq ...$).

Il prof ha dato la definizione di continous map (mappe continue) che verranno riprese nella lezione successiva.

Definizione 21 — Continuous maps

Prendiamo due PoSet X, Y e una relazione che li lega:

$< X, \underset{X}{\sqsubseteq}, \sqcup_X, \sqcap_X >< Y, \underset{Y}{\sqsubseteq}, \sqcup_Y, \sqcap_Y >, \quad f : X \to Y$

f è continua se: per tutte le catene $C \subseteq X$ valgono le seguenti proprietà:

- Esiste $\sqcup_X C$
- Esiste $\sqcup_Y f(C)$
- Esiste $\sqcup_Y f(C) = f(\sqcup_X C)$

4 Lezione 4 del 30 settembre 2026

In questa lezione vedremo i punti fissi (fix points), iterazioni e teoremi legati a questi argomenti.

4.1 Punti fissi (Fixed points)

Iniziamo dando una definizione di punto fisso per una funzione f.

Definizione 22 — Punti fissi

Data una $f : X \to X, x \in X$ è un punto fisso se $f(x) = x$

Possono esserci più punti fissi all'interno di una funzione.

Indichiamo con $\text{Fix}(f)$ l'insieme dei punti fissi per la funzione f, definiamo:

$\text{Fix}(f) = \{x \mid f(x) = x\}$

Possiamo definire anche i **least** e **greatest** fix points per una funzione f:
- $x \in \text{Fix}(f)$ è il **least fixed point (lfp)** se, $\forall y \in \text{Fix}(f) x \sqsubseteq y$
- $x \in \text{Fix}(f)$ è il **greatest fixed point (gfp)** se, $\forall y \in \text{Fix}(f) x \sqsupseteq y$

Per i nostri scopi di analisi statica siamo interessati a trovare **lfp** e **gfp**. Possono non esistere, ma nel caso in cui dovessero esserci sono unici sotto determinate condizioni.

Diamo un'occhiata a un esempio con un reticolo:

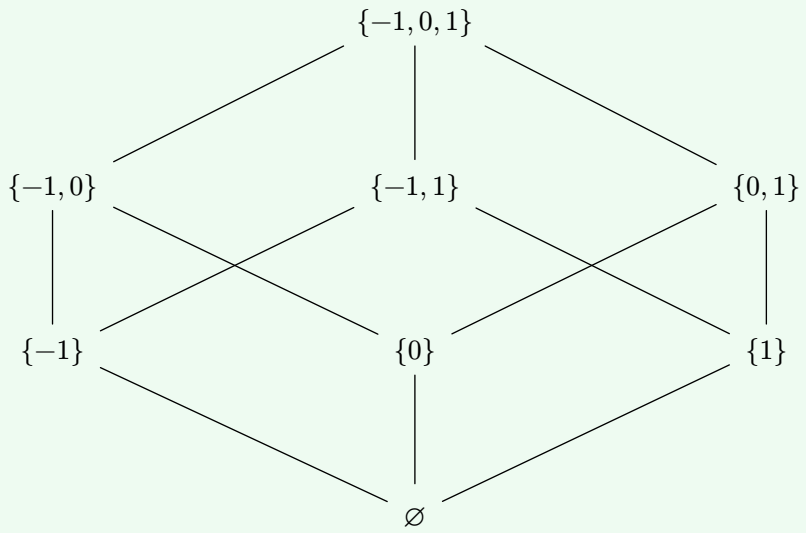
Esempio 11

Prendiamo in considerazione il PoSet $< \mathcal{P}(\{-1, 0, 1\}), \subseteq >$, indichiamo con $X = \{-1, 0, 1\}$

E definiamo una funzione $f(x)$ che lavora sull'insieme delle parti del PoSet:

$$f(x) = \begin{cases} x \setminus \{1\} & \text{se } 1 \in X \\ x \cup \{-1\} & \text{altrimenti} \end{cases}$$

Prendiamo in considerazione il reticolo per $\mathcal{P}(X)$



Dobbiamo trovare i **punti fissi** di questo reticolo, ovvero insiemi dove vale $f(x) = x$. Guardiamo la definizione di $f(x)$, dobbiamo trovare degli insiemi all'interno del reticolo che dati alla funzione restituiscano lo stesso insieme.

Prendendo ogni singolo insieme abbiamo:

- $f(\{\varnothing\}) = \{-1\}$
- $f(\{-1\}) = \{-1\}$ -> **punto fisso**.
- $f(\{0\}) = \{-1, 0\}$
- $f(\{1\}) = \{-1, 1\}$
- $f(\{-1, 0\}) = \{-1, 0\}$ -> **punto fisso**.
- $f(\{-1, 1\}) = \{-1\}$
- $f(\{0, 1\}) = \{0\}$
- $f(\{-1, 0, 1\}) = \{-1, 0\}$

Abbiamo trovato $\{-1, 0\}$ e $\{-1\}$ come punti fissi della funzione $f(x)$. Questo concetto tornerà utile per analisi statica.

Un altro concetto fondamentale sono le iterazioni per una funzione $f(x)$. Possiamo definire le iterazioni su $f(x)$ nel seguente modo:

Definizione 23 — Iterazioni

Le iterazioni di f su $x \in X$ sono definite come:
- $f^0(x) = x$
- $f^{n+1}(x) = f(f^n(x))$

Anche queste iterazioni ci permettono di trovare punti fissi per i nostri scopi di analisi statica.

Introduciamo il teorema fix-point di Knaster-Tarski.

Teorema 2 — Teorema di Knaster-Tarski sui punti fissi

Prendiamo un reticolo: $< X, \sqsubseteq, \sqcup, \sqcap >$, e una funzione $f : X \to X$, monotona

Allora, $< \text{Fix}(f), \sqsubseteq, \sqcup, \sqcap >$ è un reticolo completo.

Possiamo definire anche **lfp** e **gfp**:
- $\text{lfp}(f) = \cap \{l \mid l \sqsupseteq f(l)\}$
- $\text{gfp}(f) = \cup \{l \mid f(l) \sqsupseteq l\}$

Questo ci dice che esiste un fix point per il reticolo.

Abbiamo anche il teorema di Kleene che ci dice che possiamo trovare un punto fisso all'interno di un complete lattice.

Teorema 3 — Teorema di Kleene

Sia $< X, \sqsubseteq, \sqcup, \sqcap, \bot, \top >$ un complete lattice, e sia $f : X \to X$ continua.

La funzione f ha un punto fisso, e può essere calcolato come:

$$\sqcup_{n \geq 0} \, f^n(\bot)$$

In poche parole possiamo trovare sempre un punto fisso all'interno di un reticolo (o lattice) completo dopo un numero finito di iterazioni sulla funzione f.

Aggiungere esempi fatti a lezione sul codice.

5 Lezione 5 del 1 ottobre 2026

Nella scorsa lezione abbiamo introdotto i punti fissi per una funzione f, **least fixed point** e **greatest fixed point**, teorema di Knaster-Tarski e di Kleene.

Adesso possiamo introdurre un linguaggio toy per modellare la semantica dei nostri programmi, chiameremo questo linguaggio **Imp**

5.1 Imp, linguaggio toy

Definiamo il linguaggio **imp** come una grammatica libera da contesto (cfg). Definiamo le regole per espressioni aritmetiche, booleane e statement.

$e := x \mid n \mid e_1 \text{op}_a e_2 \quad n \in \mathbb{Z}, \text{op}_a \in \{+, -, *, /\}$

$b := \text{true} \mid \text{false} \mid \neg b \mid b_1 \text{op}_b b_2 \mid e_1 \text{op}_c e_2 \quad \text{op}_b \in \{||, \&\&\}, \text{op}_c \in \{==, !=, <, >\}$

$s := x = e \mid \text{skip} \mid s_1 s_2 \mid \text{if } b \text{ then } s_1 \text{ else } s_2 \mid \text{while } b\ s$

Questa è la nostra grammatica per il linguaggio **imp**. Perché definire un linguaggio toy per rappresentare la semantica dei programmi che vogliamo analizzare? Perché il calcolo del punto fisso per una funzione f risulta molto utile per studiare la semantica dei programmi.

Ora passiamo a definire la semantica, per questo linguaggio **imp**. Ovvero spieghiamo in modo dettagliato cosa succede quando valutiamo i simboli della grammatica.

> **Nota**
>
> *Piccola digressione*
>
> Alcuni linguaggi di programmazione moderni hanno dei manuali dove viene spiegata la sintassi del linguaggio, ma la descrizione della **semantica** avviene sempre con un linguaggio naturale, questo comporta libere interpretazioni da parte di chi vuole implementare un compilatore o un interprete per un determinato linguaggio. Il problema di queste libere interpretazioni è che alcuni compilatori valutano le stesse espressioni in maniera completamente differente.
>
> Ad esempio l'interprete *Javascript* varia tra Firefox, Chrome e Safari.

Torniamo a noi, definiamo in modo preciso la nostra semantica per **imp**. Prendiamo una funzione $M : X \to \mathbb{Z}$, chiamiamo questa funzione M per memoria. Un esempio d'uso di questa funzione è: $[x \to 5; y \to 9; z \to 11]$.

Formalizziamo la semantica per le produzioni per il simbolo delle espressioni aritmetiche e:

- $[\![\quad]\!]_e : e \times M \to \mathbb{Z}$
- $[\![x]\!]_e^m = m(x)$
- $[\![n]\!]_e^m = n$
- $[\![e_1 \text{op}_a e_2]\!]_e^m = [\![e_1]\!]^m \text{op}_a [\![e_2]\!]^m$

Definiamo ora la semantica per le produzioni per il simbolo delle espressioni booleane b:

- $[\![\quad]\!]_b : b \times M \to \{\mathrm{TT}, \mathrm{FF}\}$
- $[\![\mathrm{true}]\!]_b^m = \mathrm{TT} \quad [\![\mathrm{false}]\!]_b^m = \mathrm{FF}$
- $[\![\neg b]\!]_b^m = ![\![b]\!]_b^m$
- $[\![b_1 \ \&\& \ b_2]\!]_b^m = [\![b_1]\!]_b^m \overset{\wedge}{\vee} [\![b_2]\!]_e^m$
- $[\![e_1 \mathrm{op}_c e_2]\!]_b^m = [\![e_1]\!]_b^m \mathrm{op}_c [\![e_2]\!]_e^m$

Definiamo infine la semantica per il simbolo degli statement s:

- $[\![\quad]\!] : s \times M \to M$ -> questa semantica **produce** side effect.
- $[\![x = e]\!]^m = m[x \leftarrow [\![e]\!]_e^m]$ -> viene valutata e, il suo risultato viene salvato nella memoria di x.
- $[\![\mathrm{skip}]\!]^m = m$ -> non altera lo stato della memoria, è un **nop** (No Operation).
- $[\![s_1 s_2]\!]^m = [\![s_2]\!]([\![s_1]\!]^m)$
- $[\![\text{if } b s_1 \text{ else } s_2]\!]^m = \begin{cases} [\![s_1]\!]^m & \text{se } [\![b]\!]_b^m = \mathrm{TT} \\ [\![s_2]\!]^m & \text{altrimenti} \end{cases}$
- $[\![\text{while } b \ s]\!]^m = \begin{cases} [\![s \text{ while } b \ s]\!]^m & \text{se } [\![b]\!]_b^m = \mathrm{TT} \\ m & \text{altrimenti} \end{cases}$

Esistono diversi tipi di semantiche, abbiamo la big-step che calcola direttamente il risultato di un programma o espressione. Ci sono le semantiche small-step, che mostrano gli step intermedi per arrivare al risultato e semantiche notazionali.

Aggiungere esempi sulle slide fatte a lezione.

6 Lezione 6 del 2 ottobre 2026

La scorsa lezione abbiamo parlato delle tracce d'esecuzione di un programma, siamo interessati alle tracce che ci permettono di eseguire l'intero programma e di mantenere anche informazioni su tutti i passi intermedi all'interno di ogni traccia.

Prendiamo in considerazione il seguente pezzo di codice:

```
① i = 0;
② while(i < x)
③   i = i + 1;
④
```

Il punto 0 del programma ha come **stato iniziale** $[x \to 2]$, mentre il punto 4 è la terminazione.

Proviamo a scrivere le tracce d'esecuzione per ogni punto del programma:

- ①, avremo: $\{[x \to 2]^0 \to [x \to 2; i \to 0]^1\}$.
- ③, avremo: $\{[x \to 2]^0 \to [x \to 2; i \to 0]^1 \to [x \to 2; i \to 1]^3, [x \to 2]^0 \to [x \to 2; i \to 0]^1 \to [x \to 2; i \to 1]^3 \to [x \to 2; i \to 2]^3\}$

Questa che abbiamo appena descritto è la **semantica delle tracce**, oltre ad avere ogni singolo valore per ogni variabile in un determinato punto del programma, ci permette anche di mettere in evidenza le relazioni che ci sono tra le varie tracce.

Esistono anche semantiche collective, che però non permettono di rappresentare le relazioni tra le tracce d'esecuzione del programma. Vediamo un esempio di semantiche collettiva.

Esempio 12

	i	x
1	$\{0\}$	$\{2\}$
3	$\{1, 2\}$	$\{2\}$
4	$\{2\}$	$\{2\}$

Come già detto, da questa tabella non riusciamo a vedere le relazioni tra le varie tracce del progamma.

Ora diamo delle definizioni per la semantica delle tracce:

Definizione 24 — Semantica delle tracce o Maximal trace semantics

- Possiamo definire l'insieme delle tracce **finite** come: X^+
- L'insieme delle tracce **infinite** è dato da: X^ω
- L'insieme di **tutte** le tracce è dato da: $X^\infty : X^+ \cup X^\omega$

Possiamo vedere una traccia d'esecuzione come il prodotto cartesiano tra la memoria e la label, ovvero il punto di esecuzione del programma: $\mathbb{M} \times \mathbb{L}$

Possiamo definire anche l'insieme degli stati iniziali che sono un sotto insieme di $\mathbb{M}$: $\mathbb{I} \subseteq \mathbb{M}$

Definiamo anche una funzione F che ci permette di collezionare tracce d'esecuzione, la funzione F è definita come:

$$F : \mathcal{P}((\mathbb{M} \times \mathbb{L})^*) \to \mathcal{P}((\mathbb{M} \times \mathbb{L})^*)$$

$*$: operatore di Kleene, 0 o più ripetizioni.

Su questa funzione F vogliamo calcolare il $\text{lfp}^{\subseteq}_{\varnothing} F$. Calcoliamo il **least fixed point** di F partendo dall'insieme vuoto con operatore di ordinamento $\subseteq$.

Possiamo definire formalmente F come:

$$F = \lambda T. \quad \underbrace{\{i^0 \mid i \in I\}}_{\text{Insieme degli stati iniziali}}$$
$$\cup \underbrace{\left\{ i_0^{l_0} \to i_1^{l_1} \to \ldots \to i_{n-1}^{l_{n-1}} \to i_n^{l_n} \mid i_0^{l_0} \to i_1^{l_1} \to \ldots \to i_{n-1}^{l_{n-1}} \in T \right\}}_{\text{Insieme delle tracce non ancora terminate}}$$
$$\cup \underbrace{\left\{ i_0^{l_0} \to i_1^{l_1} \to \ldots \to i_n^{l_n} \mid i_0^{l_0} \to i_1^{l_1} \to \ldots \to i_n^{l_n} \in T \wedge i_n^{l_n} \text{ blocca con } [\![\cdot]\!] \right\}}_{\text{Insieme delle tracce bloccanti}}$$

Per far avanzare una traccia non terminata si prende la traccia interessata e si applica l'operatore: $[\![\text{next}(l_{n-1})]\!] i_{n-1} = i_n^{l_n}$ Ovvero valutiamo l'espressione corrente al punto $n-1$ per ottenere lo stato nel punto d'esecuzione n.

Riprendiamo l'esempio di codice che abbiamo visto all'inizio di questa lezione e calcoliamo $\text{lfp}^{\subseteq}_{\varnothing} F$ su quel programma.

Esempio 13

```
① i = 0;
② while(i < x)
③ i = i + 1;
④
```

Abbiamo come stati iniziali $I = \{[x \to 2]\}$. Partendo da I calcoliamo le tracce d'esecuzione per il programma.

1. $F^1 = F(\varnothing) = \{[x \to 2]^0\}$, per far evolvere questa traccia facciamo $\text{next}(0) = [\![i \to 0]\!]^1_0$ quindi otteniamo: $[\![i = 0]\!][x \to 2] = [x \to 2; i \to 0]^1$. Quindi la seconda iterazione avremo:
2. $F^2 = F(F^1) = F(\{[x \to 2]^0\}) = \{[x \to 2]^0, [x \to 2]^0 \to [x \to 2; i \to 0]^1\}$.

L'operatore $\text{next}(l)$ applicato a tutte le tracce in input che possono evolvere. Alla terza iterazione avremo:

3. $F^3 = F(F^2) = F(\{[x \to 2]^0, [x \to 2]^0 \to [x \to 2; i \to 0]^1\})$; applichiamo il next$(n)$ a tutte le tracce in input:

per la traccia $[x \to 2]^0$ otteniamo lo stesso risultato di F^2, quindi applichiamo next(1) sulla nuova traccia $[x \to 2; i \to 0]^1$ Così facendo abbiamo: next(1) = i++ -> $[\![i{+}{+}]\!]$ $[x \to 2; i \to 0] = [x \to 2; i \to 1]$. Infine otteniamo $F^3 = \{[x \to 2]^0, [x \to 2]^0 \to [x \to 2; i \to 0]^1, [x \to 2]^0 \to [x \to 2; i \to 0]^1 \to [x \to 2; i \to 1]^3\}$

Piccola precisazione: Perché non valutiamo la guardia all'interno del ***while*** $(i < x)$*? Non viene valutata perché l'operatore next(l) tiene già conto se dovessi rimanere dentro al* ***while*** *o uscirne fuori. In poche parole next(l) gestisce la logica di un CFG (Control Flow Graph).*

Andiamo avanti con la quarta iterazione:

4. $F^4 = F(F^3) = F(\{[x \to 2]^0, [x \to 2]^0 \to [x \to 2; i \to 0]^1, [x \to 2]^0 \to [x \to 2; i \to 0]^1 \to [x \to 2; i \to 1]^3\})$

Stesso discorso di prima, dobbiamo far evolvere tutte le tracce in input, prendiamo l'ultima traccia di F^3 che è quella che ci permette di ottenere un nuovo risultato.

Dobbiamo applicare next(3) = i++ -> $[\![i++]\!][x \to 2; i \to 1]^3 = [x \to 2; i \to 2]^3$ Così facendo otteniamo $F^4 = \{[x \to 2]^0, [x \to 2]^0 \to [x \to 2; i \to 0]^1, [x \to 2]^0 \to [x \to 2; i \to 0]^1 \to [x \to 2; i \to 1]^3\}, [x \to 2]^0, [x \to 2]^0 \to [x \to 2; i \to 0]^1, [x \to 2]^0 \to [x \to 2; i \to 0]^1 \to [x \to 2; i \to 1]^3\} \to [x \to 2; i \to 2]^3\}$

Passiamo alla quinta iterazione.

5. $F^5 = F(F^4)$, possiamo vedere questa operazione come l'unione di F^{l-1} con la nuova traccia calcolata tramite next$(l-1)$. Quindi prendiamo l'ultima traccia di F^4 e calcoliamo next(3) = **skip** -> $[\![\text{skip}]\!][x \to 2; i \to 2]^4$.

Otteniamo quindi: $F^5 = F^4 \cup \{[x \to 2]^0 \to [x \to 2; i \to 0]^1 \to [x \to 2; i \to 1]^3 \to [x \to 2; i \to 2]^3 \to [x \to 2; i \to 2]^4\}$

Infine alla sesta iterazione troviamo il fix point di F.

6. $F^6 = F(F^5) = F^5$, trovato il **fix point** di F.

Se guardiamo all'iterazione 5 e 6, possiamo vedere che il fix point era stato trovato all'iterazione numero 5, però tecnicamente non ne eravamo a conoscenza, il teorema di Kleene per trovare i punti fissi ci dice di fermarci finché non troviamo $F^n = F(F^{n-1}) = F^{n-1}$. Siamo riusciti a trovare un fix point per questo programma perché è un programma costante, se non fossimo stati a conoscenza della guardia dentro il while il problema sarebbe stato indecidibile.

7 Lezione 7 del 7 ottobre 2026

Abbiamo visto nella scorsa lezione la semantica delle tracce che non sempre è calcolabile per tutti i programmi. Avevamo accennato anche alla **collective semantics**, che ci permette di tenere informazioni sui valori che possono assumere le variabili ad ogni punto d'esecuzione del programma.

Per ottenere i dati per la collective semantics, prendo tutte le memorie al punto n di ogni traccia che ho ottenuto dalla trace semantics e unisco i valori tra loro.

Se prendiamo in esempio il codice che abbiamo visto la scorsa lezione:

Esempio 14

```
① i = 0;
② while(i < x)
③ i = i + 1;
④
```

- Per il punto ⓪ avremo $[x \to 2]$, che era definito come stato iniziale.
- Per il punto ① avremo $\{[x \to 2, i \to 0]\}$.
- Per il punto ③ avremo $\{[x \to 2, i \to 1]; [x \to 2, i \to 2]\}$.

Come già detto nella scorsa lezione, con la collective semantics perdiamo la relazionalità tra le tracce, dal punto ① al punto ③ non riusciamo a ricostruire correttamente la traccia originale.

Passiamo ora a definire formalmente la collective semantics.

7.1 Collective semantics

- $[\![\cdot]\!]_c \mathcal{P} \to \mathcal{P}$; semantica collettiva per gli statement.
- $[\![\text{skip}]\!]_c\ \mathcal{M} = \mathcal{M}$
- $[\![s_0\ s_1]\!]_c\ \mathcal{M} = [\![s_1]\!]_c([\![s_0]\!]_c \mathcal{M})$
- $[\![x = e]\!]_c\ \mathcal{M} = \{m[x \leftarrow [\![e]\!]^m] \mid m \in M\}$; $[\![e]\!]^m$ è la semantica dell'interprete e non la collecting.
- $[\![\text{if } b \text{ then } s_0 \text{ else } s_1]\!]_c \mathcal{M} = [\![s_0]\!]_c \mathcal{M}\ |_b \cup [\![s_1]\!]_c \mathcal{M}\ |_{!b}$
- $[\![\text{while } b\ s]\!]_c \mathcal{M} = \left(\cup_{i \geq 0} \left([\![s]\!]_c \cdot |_b\right)^i \mathcal{M}\right) |_{!b} \equiv \left(\text{lfp}^{\subseteq}_{M_0}\ F\right) |_{!b}$

L'operatore $|_b$ vuol dire filtrare da b positivo (!b vuol dire filtrato da not b), definendo formalmente otteniamo: $\mathcal{M}\ |_b = \{m \mid [\![b]\!]^m = \text{TT}\}$

La semantica del **while** è un po' complessa, cerchiamo di spiegarla a parole. La sintassi del **while** restituisce in output il set delle memorie dove b è falso. Tutti gli insiemi per cui b è positivo rimangono all'interno del ciclo **while**.

Vediamo un esempio per l'**if**:

Esempio 15

Abbiamo il seguente set di memorie: $\{[x \to 3, y \to 1], [x \to 6, y \to 2], [x \to 9]\}$

Prendiamo il seguente codice come esempio:

```
① if (x < 5)
②   y = 10
③ else
④   y = 11
⑤
```

Applichiamo la **collective semantics** per l'**if** e otteniamo il nuovo set di memorie:

$\{[x \to 3, y \to 10], [x \to 6, y \to 11], [x \to 9, y \to 11]\}$

7.2 Analisi Data Flow (Data Flow Analysis, DFA); Slide 04

Il prof ha iniziato a riprendere concetti di Data Flow Analysis, il punto principale non è saper fare DFA ma capire cosa comporta la DFA quando vogliamo fare interpretazione astratta.

📝 **Possibile domanda d'esame**

Capire cosa comporta l'analisi data flow per l'interpretazione astratta.

In interpretazione astratta possiamo fare analisi con domini non ACC (Ascending Chain Condition).

🔁 **Richiamo — Ascending Chain Condition**

Come visto in Sezione 3.2, l'**Ascending Chain Condition** ci dice che possiamo trovare un punto fisso per catene ascendenti infinite perché tendono a stabilizzarsi su un punto fisso.

Nell'analisi data flow usiamo i CFG, **Control Flow Graph**, per rappresentare il flusso d'esecuzione di un programma. L'analisi DFA viene usata dai compilatori per ottimizzare il codice in output.

Per esempio un'ottimizzazione fatta tramite DFA è quella delle **available expression**.

Definizione 25 — Available expression

Tipo di ottimizzazione dove il compilatore invece di ricalcolare la stessa espressione controlla se sia già salvata in memoria, e soprattutto che **non sia stata modificata** durante l'esecuzione

dal punto in cui viene calcolata al punto in cui vogliamo usarla, questo su **tutti** i percorsi del programma, evitando di doverla ricalcolare in altri punti del programma.

Utile per:

- Evitare di valutare la stessa espressione più volte.
- Salvare il valore in un registro temporaneo.
- Quando x viene assegnato, tutte le espressioni contenenti x non sono più disponibili (killed).
- Quando x viene assegnato, tutte le espressioni calcolate diventano available (gen).

Noi ci concentreremo su espressioni aritmetiche.

8 Lezione 8 dell'8 ottobre 2026

La scorsa lezione abbiamo introdotto le analisi DFA e l'analisi delle available expressions, riprendiamo da lì definendole formalmente e guardiamo un esempio.

8.1 Available expressions, pt. 2

Definizione 26 — Available expression, formalizzazione

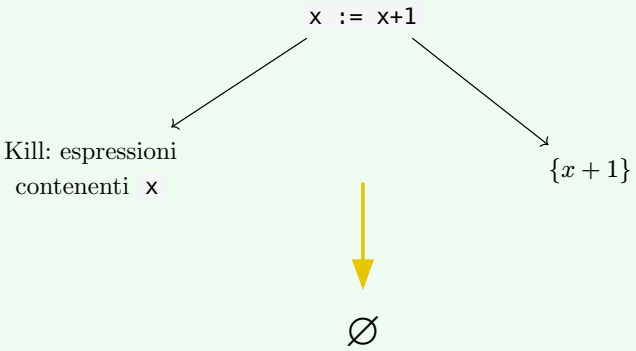
È un tipo di analisi **forward** e definita.

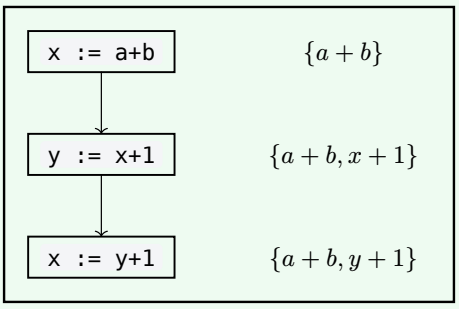
- Per ogni program point, nel nostro caso per ogni blocco, teniamo traccia dell'insieme delle available expressions.
- Indichiamo con $\text{AExp}(e)$, l'insieme delle espressioni aritmetiche non banali in e.
- Indichiamo con $\text{AExp}(b)$, l'insieme delle espressioni aritmetiche non banali in b.
- Indichiamo con $\text{FV}(a)$, l'insieme delle variabili libere, free-variables, in a.

Definiamo **kill** e **gen**:

$$\textbf{kill}(x := e) = \{a \in \text{AExp}(e) \mid x \in \text{FV}(a)\} \quad \textbf{gen}(x := e) = \{a \in \text{AExp}(e) \mid x \notin \text{FV}(a)\}$$

Esempio 16





Come possiamo vedere, nel caso $x := x + 1$ l'insieme **gen** è vuoto, questo perché $\text{FV}(x + 1)$ contiene la variabile x e non è compatibile con la definizione dell'insieme **gen**, di conseguenza deve appartenere all'insieme **kill** $= \{x\}$.

Definiamo adesso le available expression in ingresso e in uscita da un blocco.

Definizione 27 — Available expression, entry ed exit

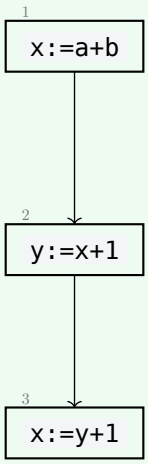
Le espressioni in ingresso verso un blocco sono le espressioni available in uscita dal blocco precedente Come già detto, l'available expression è un'analisi di tipo forward , le informazioni vengono propagate da stati di entry (ingresso) a stati d'uscita (exit).

$$\text{entry}(pp) = \text{exit}(\text{predecessor}(pp))$$

pp: program point.

Guardiamo un esempio per entry ed exit:

Esempio 17



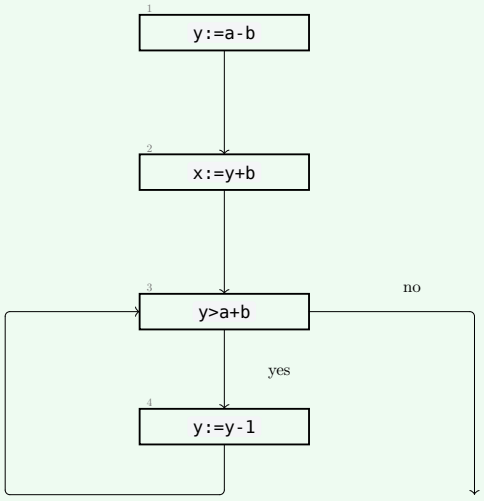
$entry(1) = \varnothing$
$exit(1) = \{a+b\}$

$entry(2) = \{a+b\}$
$exit(2) = \{a+b, x+1\}$

$exit(3) = \{a+b, y+1\}$

Per combinare queste informazioni con l'operatore di join, dobbiamo tenere conto che, in ogni punto del programma, un'espressione è available solo se è già stata calcolata, e non più modificata, su ogni possibile percorso che porta a quel punto. Guardiamo un esempio:

Esempio 18



$exit(2) = \{a-b, y+b\}$
$exit(4) = \{a-b, a+b\}$
$entry(3) = ?$

Per calcolare il valore di $entry(3)$ riprendiamo la formula che abbiamo scritto su per gli stati entranti e uscenti: $entry(pp) = \text{exit}(\text{predecessor}(pp))$.

Quindi per calcolare l'$entry(3)$ dobbiamo prendere in considerazione l'$exit$ di tutti i blocchi che entrano dentro il blocco 3. Abbiamo il blocco 2 e il blocco 4. Prendiamo $exit(2)$ e $exit(4)$ e facciamo l'intersezione e otteniamo

- $exit(2) = \{a-b, y+b\}$
- $exit(4) = \{a-b, a+b\}$
- $exit(2) \cap exit(4) = \{a-b\}$

In questo caso dentro al blocco 4 abbiamo $y := y-1$ il suo ***gen*** *è $\varnothing$, mentre il* ***kill*** *è $\{y-1, y+b\}$.*

Formalizziamo le available expression in ingresso e in uscita da un blocco i.

Definizione 28 — Available expression, exit e entry

Se vogliamo formalizzare le available expression che entrano in un blocco i possiamo farlo con:

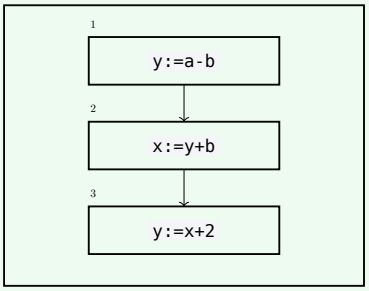
$$\mathbf{entry}(i) = \cap \{\mathbf{exit}(i') \mid (i', i) \in \mathbf{edges}(G)\}$$

Mentre per le available expression che escono da un blocco i:

$$\mathbf{exit}(i) = (\mathbf{entry}(i) \setminus \mathbf{kill}(\mathbf{stmt}(G, i))) \cup \mathbf{gen}(\mathbf{stmt}(G, i))$$

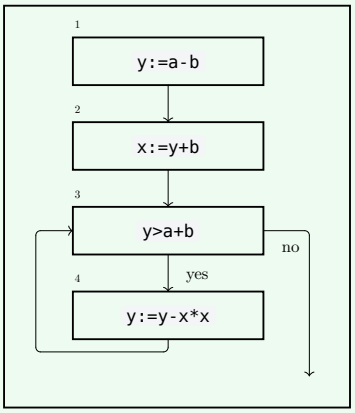
Guardiamo due esempi di calcolo per entry ed exit.

Esempio 19



Block	entry	exit
1	$\varnothing$	$(\text{entry}(1) \setminus \{a \in \text{AE} \mid y \in \text{FV}(a)\}) \cup \{a - b\}$
2	exit(1)	$(\text{entry}(2) \setminus \{a \in \text{AE} \mid x \in \text{FV}(a)\}) \cup \{y + b\}$
3	exit(2)	$(\text{entry}(3) \setminus \{a \in \text{AE} \mid y \in \text{FV}(a)\}) \cup \{x + 2\}$

Esempio 20



Block	entry	exit
1	$\varnothing$	$(\text{entry}(1) \setminus \text{kill}(y := a - b)) \cup \text{gen}(y := a - b)$
2	exit(1)	$(\text{entry}(2) \setminus \text{kill}(x := y + b)) \cup \text{gen}(x := y + b)$
3	exit(2) $\cap$ exit(4)	$(\text{entry}(3) \setminus \text{kill}(y > a + b)) \cup \text{gen}(y > a + b)$
4	exit(3)	$(\text{entry}(4) \setminus \text{kill}(y := y - x * x)) \cup \text{gen}(y := y - x * x)$

8.2 Altre analisi Dataflow

Fino ad ora abbiamo visto l'analisi delle available expressions, che è un tipo di analisi **forward** e **definita**.

- **Forward** perché l'analisi viene svolta raccogliendo informazioni seguendo il flusso originale del progamma, quindi dal blocco A al blocco B.
- **Backward**, le informazioni sono propagate dall'exit del blocco all'entry dei blocchi precedenti, un esempio è l'analisi Live variable analysis.
- **Definita, o must,** perché un'espressione è considerata available se è disponibile in ***tutti*** i cammini che arrivano a quel program point.

- **Possibile, o may**, perché un'informazione (ad esempio una variabile live o una definizione che raggiunge il punto) vale in un program point se vale in ***almeno un*** cammino che arriva a quel punto, o che parte da esso nelle analisi backward.

Altri tipi di analisi sono:

> **Definizione 29 — Live variables**
>
> Per ogni program point, calcoliamo quali variabili ***possono (may)*** essere live all'uscita da quel punto. Una variabile è detta live se il suo valore può essere letto lungo almeno un cammino che parte dal program point, senza essere prima sovrascritto.
>
> Notare che nella definizione c'è ***may***, questo ci indica che si tratta di un'analisi may e non definita; nei punti di join si fa l'unione degli insiemi.

> **Definizione 30 — Reaching definitions**
>
> Per ogni program point, calcoliamo quali assegnamenti possono (may) essere stati eseguiti e non sovrascritti quando l'esecuzione raggiunge quel punto lungo un qualche cammino.
>
> È un'analisi ***forward*** e ***may***: nei punti di join si fa l'unione degli insiemi.

> **Definizione 31 — Very busy expressions**
>
> Per ogni program point, calcoliamo quali espressioni devono (must) essere very busy all'uscita da quel punto. Un'espressione è very busy se, lungo tutti i cammini che partono da quel punto, viene usata prima che una qualsiasi delle variabili che vi compaiono sia ridefinita.
>
> È un'analisi ***backward*** e ***must***: nei punti di join si fa l'intersezione degli insiemi.

Ricapitolando velocemente:

- **Analisi Possible**: may, some paths, unione.
- **Analisi Definite**: must, all paths, intersezione.
- **Analisi Forward**: fino al program point.
- **Analisi Backward**: dal program point a ritroso.

8.3 Lattice in analisi dataflow

Possiamo vedere le analisi dataflow come dei lattice, o reticoli. **TUTTE** le analisi dataflow sono dei ***complete lattice*** e devono rispettare l'ACC Sezione 3.2.

Possiamo definire l'analisi dell'available expression come un lattice:

- $< \mathcal{P}(AE), \supseteq, AE, \varnothing, \cap, \cup >$

Per le live variable:

- $< \mathcal{P}(Var), \subseteq, \varnothing, VAR, \cap, \cup >$

Dato che possiamo vedere le analisi dataflow come reticoli, teniamo in considerazione che per le analisi definite, o must, salendo lungo il reticolo perdiamo informazioni.

Ricordiamo che il valore $\bot$ rappresenta uno stato di errore o di assenza di informazioni, mentre il valore $\top$ rappresenta lo stato ***non lo so***.

8.4 Analisi distributive

Le analisi distributive sono semplici. Ci permettono di calcolare ***COME*** avviene la computazione, per esempio l'assegnamento di variabili, calcolo di espressioni, e non ***COSA*** il programma calcola.

Definizione 32 — Analisi distributiva

Prendiamo in considerazione un lattice $< \mathcal{P}(X), \subseteq, \varnothing, X, \cup, \cap >$ e $f : \mathcal{P}(X) \to \mathcal{P}(X)$

La funzione f è distributiva se: $f(x \cup y) = f(x) \cup f(y)$

Tutte le analisi dataflow sono distributive, anche quando il lub è $\cap$.

Le analisi più complesse non sono distributive, e spesso non possono essere formalizzate come analisi dataflow. Per questo motivo è stata introdotta l'interpretazione astratta.

8.5 Propagazione di costanti

Definizione 33 — Propagazione di costanti

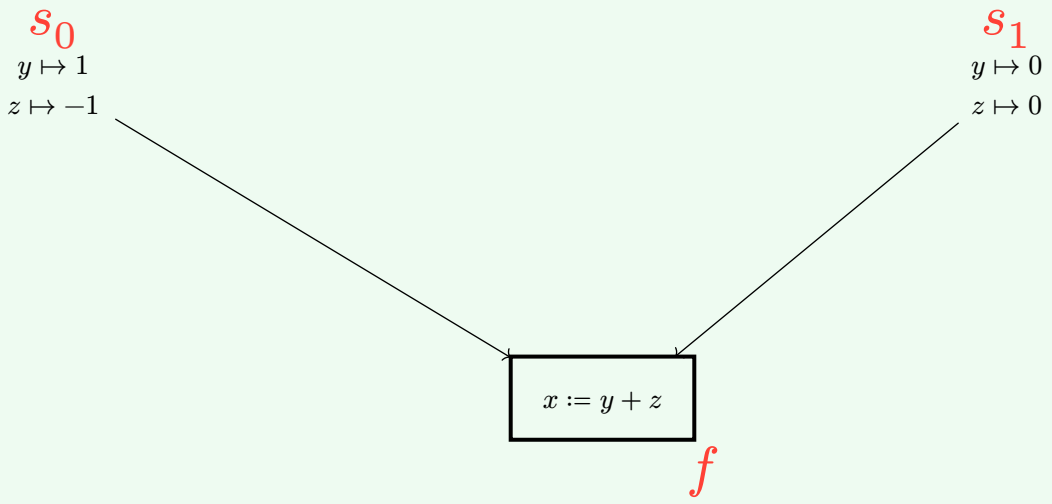
Per ogni punto del programma, determinare se una variabile ha o meno un valore costante ogni volta che l'esecuzione raggiunge quel punto.

- ***NON*** è un tipo di analisi distributiva.

Il lattice della CP è: $< \{\bot\} \cup \mathbb{Z} \cup \{\top\}, \sqsubseteq, \sqcup, \sqcap, \bot, \top >$

Guardiamo un esempio dove proviamo a dimostrare che la constant propagation sia un'analisi distributiva.

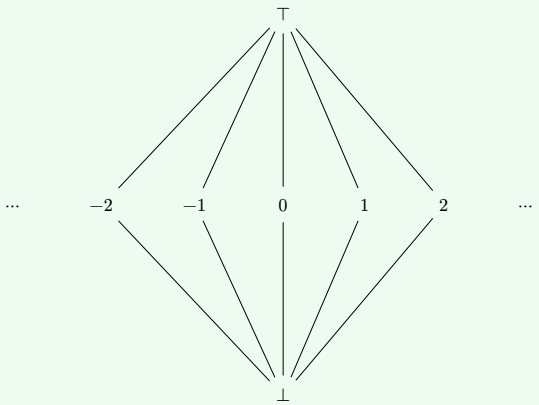
Esempio 21



Proviamo a dimostrare che constant propagation sia una funzione distributiva, abbiamo f e deve valere la proprietà che f sia distributiva, quindi:

$$f(x \sqcup y) = f(x) \sqcup f(y)$$

Possiamo vedere l'insieme dei possibili valori come un lattice su cui poi dobbiamo calcolare il lub ($\sqcup$) tra x e y.



Partiamo calcolando $f(s_0 \sqcup s_1)$, applichiamo il lub che va calcolato per ogni coppia di variabile, vediamo nel reticolo che il lub per s_0 e s_1 è $\top$, quindi otteniamo:

$$f([y \mapsto \top, z \mapsto \top]) = [x \mapsto \top, y \mapsto \top, z \mapsto \top]$$

Ricordiamo che usiamo $\top$ per indicare che non sappiamo il valore indicato. Andiamo avanti calcolando la seconda parte dell'equazione:

$$f(s_0) \sqcup f(s_1) = [x \mapsto 0, y \mapsto 1, z \mapsto -1] \sqcup [x \mapsto 0, y \mapsto 0, z \mapsto 0] = [x \mapsto 0, y \mapsto \top, z \mapsto \top]$$

Confrontiamo i risultati:

$$[x \mapsto \top, y \mapsto \top, z \mapsto \top] \neq [x \mapsto 0, y \mapsto \top, z \mapsto \top]$$

Possiamo vedere che la proprietà che f sia distributiva non viene rispettata, quindi la constant propagation non è un'analisi di tipo distributivo.

9 Lezione 9 del 9 ottobre 2026

Abbiamo visto fino ad ora la maximal trace semantics, che in un mondo ideale sarebbe quella che vorremmo calcolare, ma in realtà è infinita e indecidibile. Abbiamo visto anche la collective semantics, che è un astrazione della maximal trace semantics, dato che perdiamo la relazione tra le tracce. Di conseguenza ci serve un'approssimazione che ci permette rispondere con certezza a determinate proprietà di un programma che vogliamo studiare.

9.1 Approssimazioni

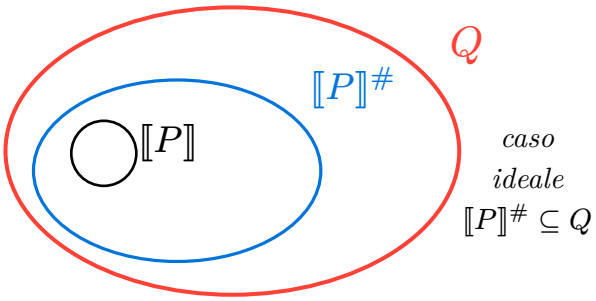
Ipotizziamo di avere una proprietà Q tale che $[\![P]\!] \subseteq Q$, generalmente questo è indecidibile.

Di conseguenza ci serve un'approssimazione $[\![P]\!]^\#$ di $[\![P]\!]$ in modo da ottenere:

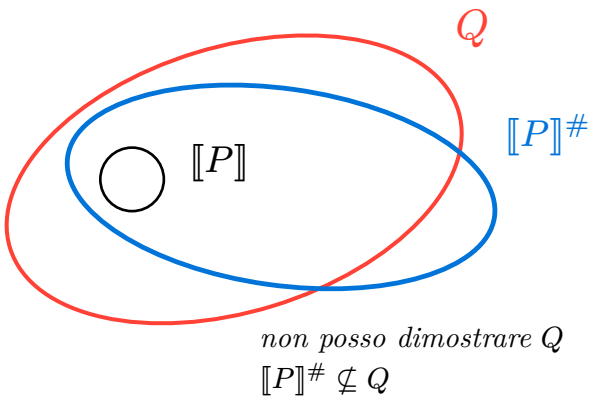
- **Soundness**: $[\![P]\!] \subseteq [\![P]\!]^\#$.
- **Decidability**: $[\![P]\!]^\# \subseteq Q$

La soundness per noi è cruciale dato che ci permette di sfruttare la decidibilità di un'approssimazione. I comportamenti originali del programma **DEVONO** essere contenuti nella nostra approssimazione $[\![P]\!]^\#$ per ottenere la soundness.

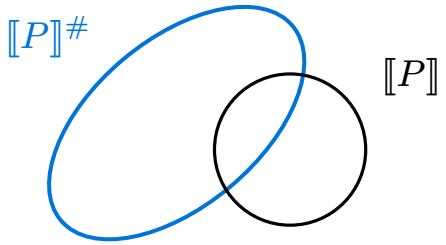
- Il caso ideale che ci permette di dimostrare che la proprietà Q vale per il programma che stiamo analizzando è quando la nostra approssimazione è contenuta in Q: $[\![P]\!]^\# \subseteq Q$



- Nel caso in cui la nostra approssimazione non sia contenuta in Q non possiamo dimostrare la proprietà Q:



- Invece nel diagramma qui sotto abbiamo che la nostra approssimazione non rispetta la proprietà di **soundness**:



$\rightarrow$ *non è sound*
abbiamo falsi negativi

In queste approssimazioni è normale perdere precisione, soprattutto nell'ultimo caso dove possiamo avere dei falsi positivi, ovvero punti che appartengono all'astrazione $[\![P]\!]^{\#}$ e si intersecano con la semantica concreta del programma $[\![P]\!]$. Quello che non possiamo accettare sono i falsi negativi, in questo caso i punti della semantica concreta che non si intersecano con la semantica astratta, questo ci porta a perdere la proprietà di ***soundness***.

Ora diamo una definizione di ***astrazione***.

Definizione 34 — Astrazione

L'***astrazione*** è il processo di sostituire qualcosa di concreto con una descrizione che riguarda alcune delle sue proprietà.

L'***astrazione*** ci permette di descrivere con precisione alcune proprietà, ma non tutte.

Un'***astrazione*** in $\mathcal{P}(\Sigma)$ di oggetti contenuti in Σ è $A \subseteq \mathcal{P}(\Sigma)$ tale che:
- Gli elementi in A sono quelli descritti in maniera precisa dall'***astrazione***, quindi senza perdita di informazioni
- Gli elementi non presenti in A devono essere rappresentati da altri elementi di A, perdendo precisione.

Quando analizziamo dei programmi dobbiamo considerare i vari oggetti che possono rappresentare lo stato della computazione:
- Valori: booleani, interi con segno, etc …
- Variabili
- Environment
- Stack, etc…

Le proprietà sono un insieme di oggetti, che hanno quella proprietà:
- Numeri naturali dispari $= \{1, 3, 5, 2n + 1, ...\}$
- Numeri pari interi $= \{2z \mid z \in \mathbb{Z}\}$

9.1.1 Insieme delle proprietà

L'insieme delle proprietà $\mathcal{P}(\Sigma)$ di oggetti Σ è un lattice completo distributivo, possiamo definirlo nel seguente modo:

$$< \mathcal{P}(\Sigma), \subseteq, \varnothing, \cup, \cap, \neg >$$

- $\varnothing$, è il valore bottom, se otteniamo $\varnothing$ vuol dire che la proprietà non è rispettata da nessun programma.
- Σ, è il valore di top, vuol dire che il programma rispetta tutte le proprietà.

- $\subseteq$, l'operatore di ordinamento ci indica che salendo su per il reticolo avremo proprietà sempre più astratte.
- $\neg$, è la negazione delle proprietà: $\neg P = \Sigma \setminus P$.

Vediamo un esempio veloce per la negazione:

Esempio 22

Prendiamo l'insieme degli oggetti: $\Sigma = \{1, 2, 3, 4, 5, 6\}$

Definiamo la proprietà di essere pari come:

$$P = \{2, 4, 6\}$$

La proprietà negata, $\neg P$ è data da:

$$\neg P = \Sigma \setminus P$$

$$\neg P = \{1, 2, 3, 4, 5, 6\} \setminus \{2, 4, 6\}$$

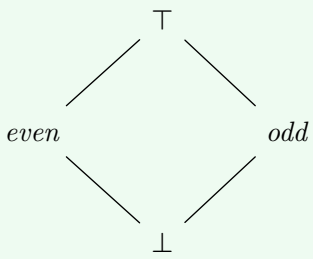
Quindi otteniamo:

$$\neg P = \{1, 3, 5\}$$

Esempio 23

Ipotizziamo di voler fare l'analisi di parità su un programma.

Il complete lattice della parità possiamo definirlo come:



Prendiamo il caso $x \to (\text{Even}, [1; 5])$, sapendo che x deve assumere un valore pari, possiamo ridurre il suo intervallo a: $x \to (\text{Even}, [2; 4])$.

Notare che il 3, anche se è dispari, rimane nell'intervallo, nonostante siamo a conoscenza del fatto che x sia un numero pari. Questa analisi ci permette di ottenere il ***miglior intervallo possibile***, che ovviamente può contenere del rumore, in questo caso il numero 3.

Il reticolo rappresenta il dominio dell'analisi.

9.1.2 Direzione dell'astrazione

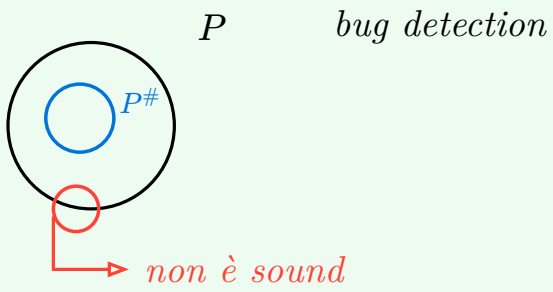
Quando vogliamo approssimare una proprietà P che ci interessa con un'astrazione $P^{\#}$ possiamo avere due casi:

- Un'approssimazione **dal basso** (from below): dove $P^{\#} \subseteq P$
- O un'approssimazione **dall'alto** (from above): dove $P \subseteq P^{\#}$

L'approssimazione dal basso è utile per bug detection, però perdiamo alcuni comportamenti concreti del programma nella nostra astrazione. Le due nozioni sono duali.

Guardiamo alcuni esempi di approssimazioni from below e from above.

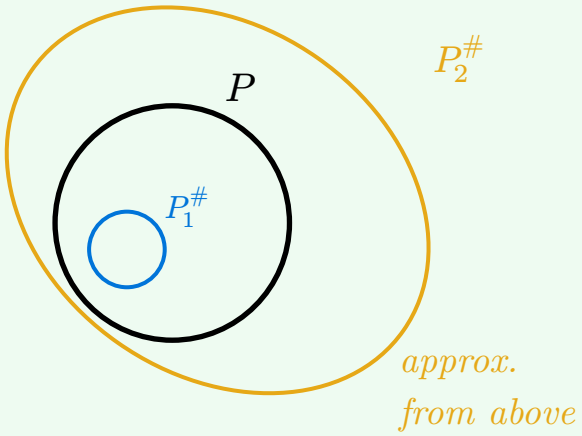
Esempio 24



Questo tipo di analisi from below è utile per bug detection, attenzione che se dovessimo avere un bug che si trova in P, ma non è dentro l'insieme della nostra astrazione $P^{\#}$, perdiamo la possibilità di trovare quel bug con la nostra approssimazione.

Se un determinato comportamento che analizziamo è dentro la nostra approssimazione $P^{\#}$ allora il programma avrà quel comportamento, altrimenti non possiamo dirlo con certezza.

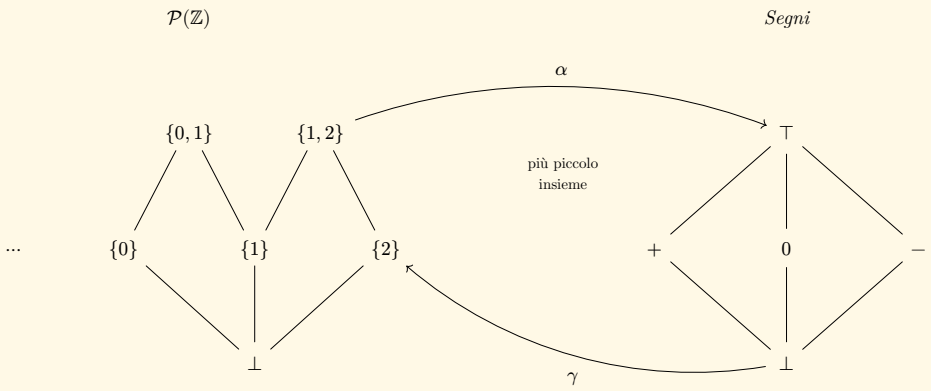
Nel caso in cui $P^{\#}$ fosse l'insieme rosso, non sarebbe un'approssimazione che rispetta la soundness.



In questo caso abbiamo un'astrazione from above, e vediamo che la nostra astrazione $P_2^{\#}$ contiene P, questo ci porta a perdere precisione ma questo è un compromesso quando facciamo una qualunque tipo di astrazione.

Nota

Possiamo vedere il passaggio da una proprietà concreta a un'approssimazione come una funzione α che mappa le proprietà dal concreto alla nostra approssimazione.



In questo diagramma abbiamo 2 reticoli, uno per l'insieme delle parti per $\mathbb{Z}$, dall'altro abbiamo il reticolo dei segni. Abbiamo una funzione α che mappa dal reticolo delle parti di $\mathbb{Z}$ nel reticolo dei segni. La funzione α deve mappare il più piccolo elemento nel reticolo dei segni che può rappresentare l'elemento che scegliamo dal reticolo dell'insieme delle parti di $\mathbb{Z}$.

Questo prende il nome di ***connessione di Galois***.

Il nostro obiettivo è ottenere la miglior approssimazione possibile per P, lo vedremo più avanti quando inizieremo a parlare di interpretazione astratta.

9.1.3 Altri tipi di approssimazioni

Esistono anche altri tipi di approssimazioni, come gli intervalli, ottagoni, poliedri e le simple congruences.

9.1.3.1 Intervalli

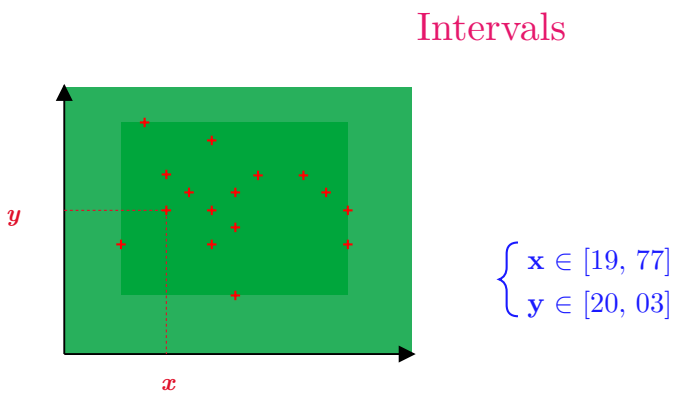


Figura 4. Approssimazione con intervalli

9.1.3.2 Ottagono

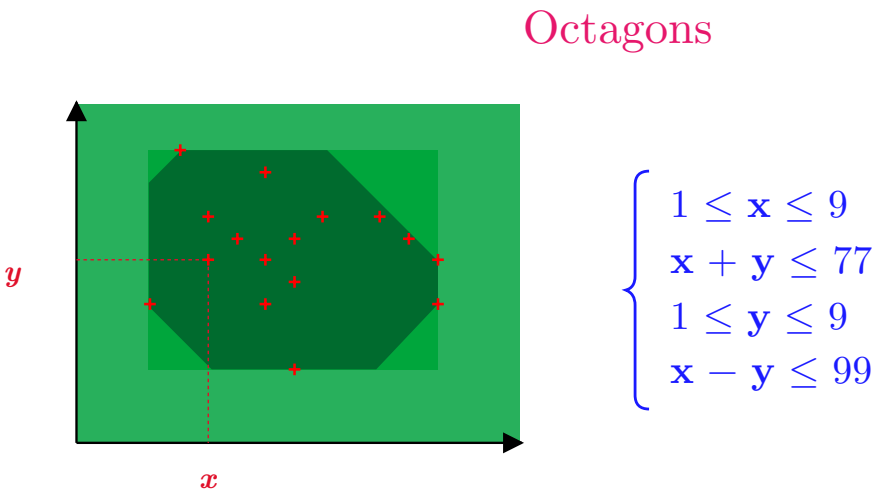


Figura 5. Approssimazione con ottagoni

È un'approssimazione più raffinata rispetto a quella degli intervalli, abbiamo dei vincoli più stringenti.

9.1.3.3 Poliedri

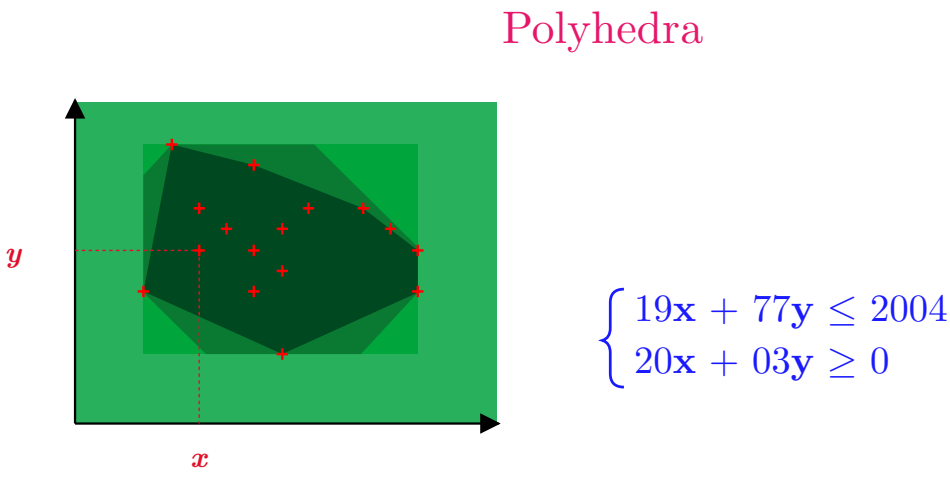


Figura 6. Approssimazione con poliedri

I poliedri hanno un problema, non esiste la funzione di astrazione α. Non esiste neanche il miglior poliedro che possa approssimare il dominio della nostra approssimazione, questo perché il dominio dei poliedri convessi non è un reticolo completo.

9.2 Conclusioni

Quando facciamo delle astrazioni, inevitabilmente perdiamo precisione aggiungendo del rumore rispetto alla semantica concreta. Per esempio con le collective semantics abbiamo delle esecuzioni spurie perché perdiamo le tracce d'esecuzione originali, se invece collezionassimo proprietà aggiungiamo degli stati spuri per via dell'approssimazione dei dati.

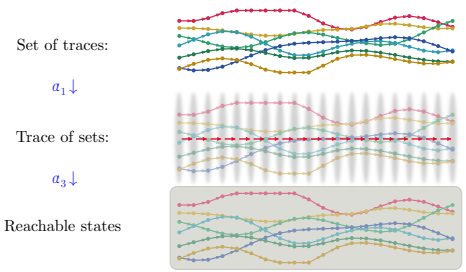


Figura 7. Astrazione in reachable states.

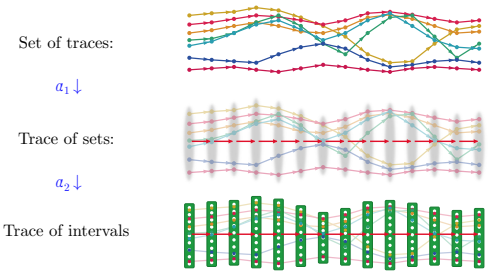


Figura 8. Astrazione in tracce d'intervalli.

Le immagini originali sono nel pacchetto di slide numero 06.

Con le nostre approssimazioni consideriamo un insieme di proprietà degli stati invece di stati concreti; approssimiamo la computazione di un programma con la collective semantics su tutti i program point. Attenzione perché anche le nostre astrazioni possono non terminare e quindi essere indecidibili.

10 Lezione 10 del 14 ottobre 2026

Nella scorsa lezione abbiamo parlato di approssimazioni e accennato alle connessioni di Galois. In questa lezione entriamo nel dettaglio per quanto riguarda le connessioni di Galois e funzioni di astrazione e concretizzazione.

10.1 Dominio concreto

Prima di definire le connessioni di Galois, definiamo cos'è un dominio concreto:

Definizione 35 — Dominio concreto

Un dominio concreto possiamo vederlo come un complete lattice:

$$< \mathcal{P}(\mathbb{Z}), \subseteq, \cap, \cup, \mathbb{Z}, \varnothing >$$

- $S_1 \subset S_2$: S_1 è più preciso di S_2.
- $S_1 \not\subseteq S_2 \land S_2 \not\subseteq S_1$: non sono comparabili.
- $S_1 \cup S_2$: x deve essere in S_1 o S_2.
- $S_1 \cap S_2$: x deve essere in S_1 o S_2.
- $\mathbb{Z}$, l'elemento top, x può essere qualunque valore.
- $\varnothing$, nessun valore possibile, o errore.

Il dominio concreto rappresenta i valori reali che assume il programma, lo vedremo come il mondo concreto da astrarre.

10.2 Dominio astratto

I domini astratti invece sono un'astrazione del dominio concreto che prendiamo in considerazione. Ci sono infiniti domini astratti che rappresentano diversi tipi di informazioni, per esempio insieme dei segni, intervalli etc. Non tutti i domini però sono confrontabili, per esempio non ha senso confrontare il dominio degli intervalli con quello dei segni dato che uno contiene più informazioni dell'altro. Di conseguenza esiste una gerarchia tra i vari domini che verrà vista più avanti nel corso.

I domini non confrontabili possiamo sfruttarli per combinarli tramite prodotto cartesiano o prodotto ridotto.

Esiste una relazione formale tra il dominio concreto C e il dominio astratto A definita come:

Definizione 36 — Dominio astratto

Abbiamo una funzione d'astrazione α che prende elementi dal mondo concreto C e li mappa nel mondo astratto A.

$$\alpha : C \to A$$

Abbiamo anche una funzione di concretizzazione, che ci permette di passare dal mondo astratto A al mondo concreto C.

$$\gamma : A \to C$$

Guardiamo ora alcuni esempi di astrazione da un dominio concreto a uno astratto.

Esempio 25

Riprendiamo la constant propagation e ridefiniamo il lattice:

$$< \mathcal{P}(\mathbb{Z}) \cup \{\top, \bot\}, \sqsubseteq_{\text{CP}}, \sqcup_{\text{CP}}, \sqcap_{\text{CP}}, \top, \bot >$$

Vediamo una porzione del reticolo della constant propagation e del suo dominio astratto:

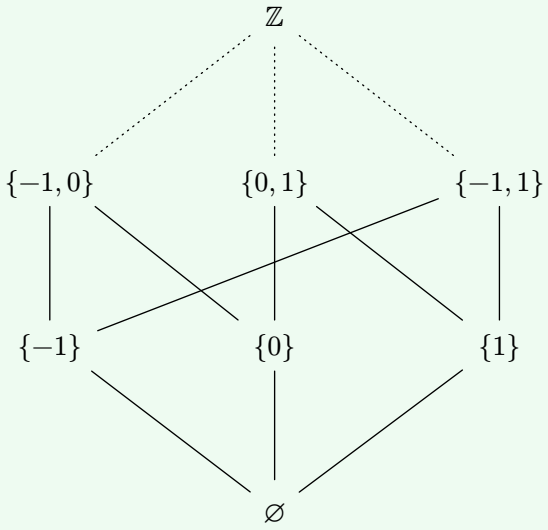


Figura 9. Reticolo Constant propagation

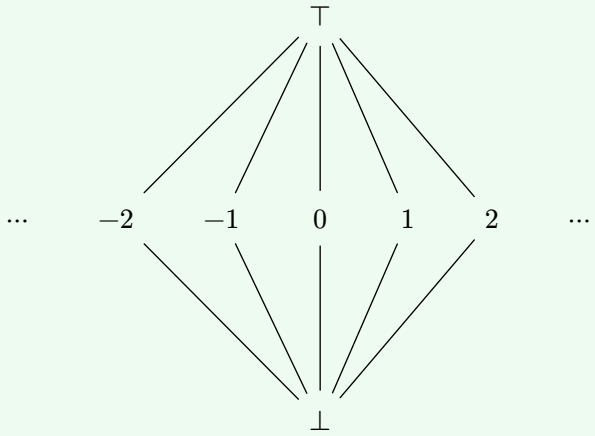


Figura 10. Reticolo dominio astratto

Ipotizziamo di star analizzando un programma e a un certo punto otteniamo la memoria $[x \to 1, y \to 0]$ e abbiamo il seguente snippet di codice da analizzare:

```
z = x / y;

// ...

x++;
```

Valutando questo snippet con la memoria di partenza otteniamo una nuova memoria $[x \to 1, y \to 0, z \to \bot]$, da questa memoria posso generare direttamente $\bot$, dato che il programma è andato in errore è inutile portarsi dietro anche il resto delle informazioni, è ovviamente una semplificazione rispetto all'informazioni che mi fornisce la memoria.

Proviamo ora a definire le funzioni α e γ per il reticolo concreto e il reticolo del dominio astratto. Partiamo dalla funzione di astrazione: $\alpha_{\text{CP}} : \mathcal{P}(\mathbb{Z}) \to \text{CP}$

Se abbiamo come argomento di α_{CP} un singoletto dell'insieme $\mathcal{P}(\mathbb{Z})$ ci basta resituire il valore di quel singoletto e mapparlo nel dominio astratto, quindi l'argomento ha cardinalità 1. Nel caso in cui dovessimo avere più elementi dentro all'argomento allora restituiremo l'elemento $\top$. In tutti gli altri casi verrà restituito l'elemento $\bot$.

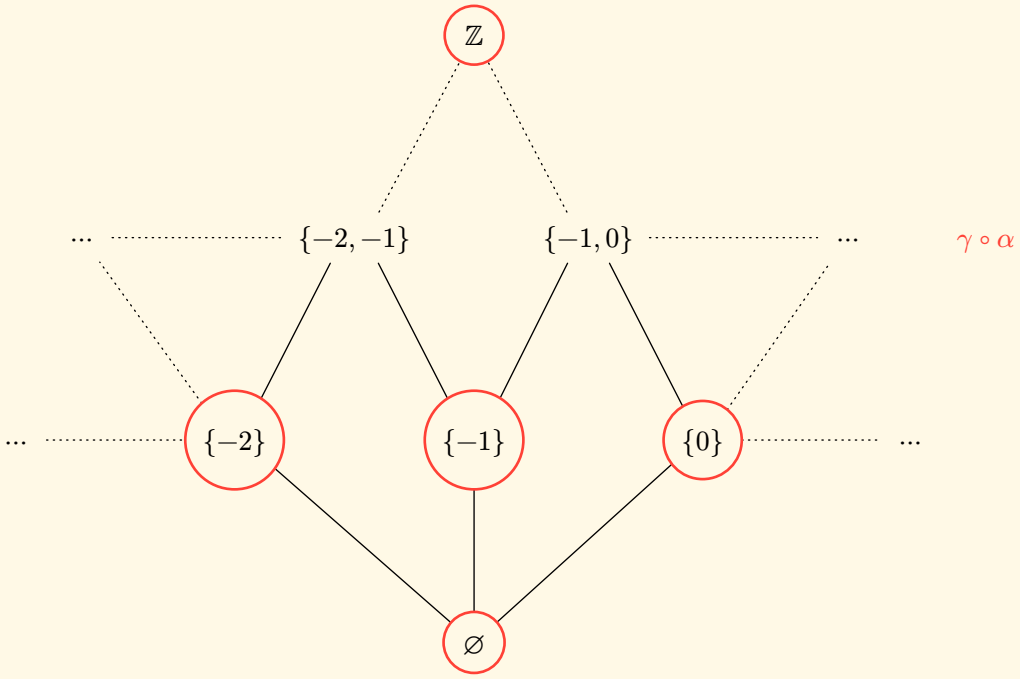
$$\alpha_{\text{CP}}(I) = \begin{cases} v & |I| = 1 \land I = \{v\} \\ \top & |I| > 1 \\ \bot & \text{altrimenti} \end{cases}$$

Come dicevamo nella scorsa lezione, quando facciamo un astrazione sul mondo concreto perdiamo informazioni, qui perdiamo la possiblità di tenere traccia se una determinata variabile può assumere più valori. Passiamo ora a definire la funzione γ_{CP}. Dobbiamo partire dal reticolo astratto e tornare al reticolo concreto. Nel caso in cui l'argomento della funzione γ_{CP} dovesse essere l'elemento $\top$, dobbiamo restituire il $\top$ del reticolo concreto, in questo caso $\mathbb{Z}$. Nel caso in cui l'argomento sia vuoto, allora restituiremo $\varnothing$. Nel caso in cui a sia un valore v, restituiremo v.

$$\gamma_{\text{CP}}(a) = \begin{cases} \mathbb{Z} & a = \top \\ \varnothing & a = \bot \\ \{v\} & a = v \end{cases}$$

Naturalmente dopo aver astratto, quando torniamo nel mondo concreto le informazioni perse non possono essere recuperate.

> **Nota**
>
> Piccola digressione...

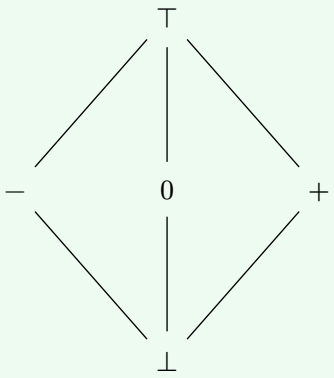


Volendo possiamo fare astrazione direttamente sul reticolo concreto come in questo caso.

Guardiamo anche un esempio per l'analisi dei segni:

Esempio 26

Partiamo definendo il dominio dei segni: $< \{0, +, -, \top, \bot\}, \sqsubseteq, \sqcup, \sqcap, \top, \bot >$



$$\alpha_{\text{sign}} : \mathcal{P}(\mathbb{Z}) \to \text{Segno}$$

$$\gamma_{\text{sign}} : \text{Segno} \to \mathcal{P}(\mathbb{Z})$$

Anche in questo caso abbiamo le funzioni di astrazione e di concretizzazione. Passiamo a definire quella di astrazione:

$$\alpha_{\text{Sign}}(I) = \begin{cases} + & \forall i \in I \quad i > 0 \\ - & \forall i \in I \quad i < 0 \\ 0 & I = \{0\} \\ \bot & I = \varnothing \\ \top & \text{altrimenti} \end{cases}$$

Mentre la funzione di concretizzazione γ_{Sign} è definita:

$$\gamma_{\text{Sign}(s)} = \begin{cases} \{0\} & S = 0 \\ \{z \mid z > 0\} & S = + \\ \{z \mid z < 0\} & S = - \\ \varnothing & S = \bot \\ \mathbb{Z} & S = \top \end{cases}$$

Questra astrazione in intervallo dei segni è **sound**, ovvero:

- Partiamo da un valore astratto.
- Concretizziamo sfruttando γ.
- Torniamo all'astrazione con la funzione α.
- È sound se e solo se il risultato è $\sqsupseteq_A$, o in generale uguale.

10.2.1 Monotonicità

Entrambe le funzioni α e γ sono monotone, α è monotona perché:

- $I_1 \subseteq I_2 \Rightarrow \alpha(I_1) \sqsubseteq_A \alpha(I_2)$
- Quindi se astraiamo qualcosa otteniamo un livello di informazione inferiore o uguale.

Mentre γ è monotona perché:

- $a_1 \sqsubseteq_A a_2 \Rightarrow \gamma(a_1) \subseteq \gamma(a_2)$
- Se concretizziamo qualcosa che rappresenta un quantitativo di informazione minore otteniamo qualcosa con meno o la stessa quantità di informazione.

10.3 Connessioni di Galois

Definiamo una connessione di Galois come:

Definizione 37 — Connessione di Galois

Presi due poset: $< C, \sqsubseteq_C >$ e $< A, \sqsubseteq_A >$, $\alpha : C \to A$ e $\gamma : A \to C$ due funzioni monotone, formano una connessione di Galois se:

$$\forall c \in C, \forall a \in A . \alpha(c) \sqsubseteq_A a \Leftrightarrow c \sqsubseteq_C \gamma(a)$$

- La prima parte ci garantisce la **soundness**: $\alpha(c) \sqsubseteq_A a \Rightarrow c \sqsubseteq_C \gamma(a)$
- La seconda parte invece ci garantisce la **best abstraction**: $c \sqsubseteq_C \gamma(a) \Rightarrow \alpha(c) \sqsubseteq_A a$

Abbiamo anche una definizione alternativa per la connessione di Galois:

Definizione 38 — Connessione di Galois, alternativa

Presi due poset: $< C, \sqsubseteq_C >$ e $< A, \sqsubseteq_A >$, $\alpha : C \to A$ e $\gamma : A \to C$ abbiamo una connessione di Galois se sono rispettate le seguenti proprietà:

- α e γ sono monotone.
- $\gamma \circ \alpha$ è estensiva; $\forall c \in C . c \sqsubseteq_C \gamma(\alpha(c))$ (quando facciamo astrazione perdiamo informazioni)
- $\alpha \circ \gamma$ è riduttiva; $\forall a \in A . \alpha(\gamma(a)) \sqsubseteq_A a$ (quando concretizziamo non perdiamo informazioni)

Esempio 27

Riprendiamo l'esempio dei segni, ricordando le definizioni delle funzioni α e γ dell'esempio precedente; prendiamo in considerazione il seguente scenario:

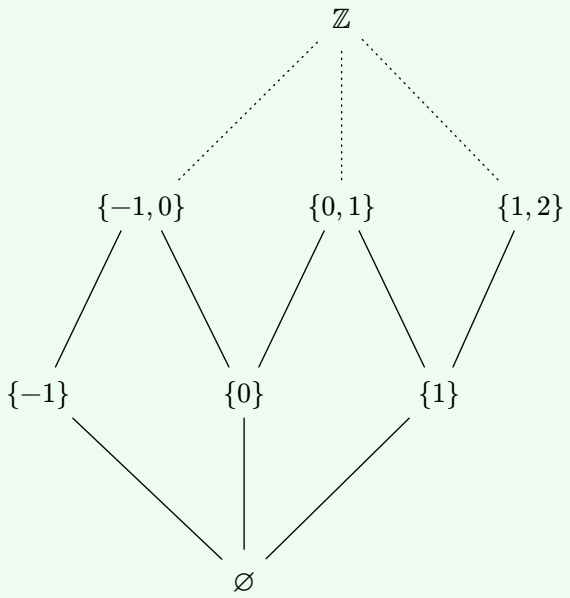


Figura 11. Reticolo concreto.

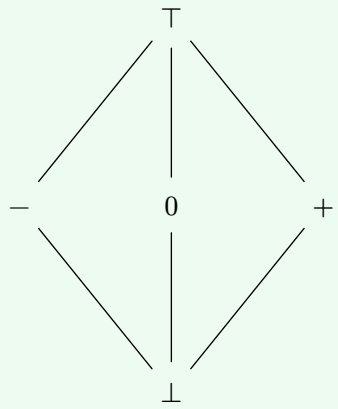


Figura 12. Reticolo dell'astrazione.

Adesso possiamo leggere la definizione di connessione di Galois, $\alpha(c) \sqsubseteq_A a \Leftrightarrow c \sqsubseteq_C \gamma(a)$, spezzandola nelle due implicazioni.

1. $\alpha(c) \sqsubseteq_A a \Rightarrow c \sqsubseteq_C \gamma(a)$

   Soundness: se l'astrazione di c sta sotto a, allora a descrive davvero c, cioè c è contenuto in ciò che a rappresenta. Esempio: $c = \{1, 2\}$ e $a = +$. Abbiamo $\alpha(c) = + \sqsubseteq_A +$ e infatti $c \subseteq \gamma(+) = \{1, 2, 3, ...\}$.

2. $c \sqsubseteq_C \gamma(a) \Rightarrow \alpha(c) \sqsubseteq_A a$

   Best astraction: se a descrive c, allora $\alpha(c)$ è almeno altrettanto preciso di a, cioè $\alpha(c)$ è la migliore astrazione di c. Esempio: $c = \{1, 2\}$ e $a = \top$. Abbiamo $c \subseteq \gamma(\top) = \mathbb{Z}$ e infatti $\alpha(c) = + \sqsubseteq_A \top$.

   Se invece avessimo (per assurdo) $\alpha(\{1, 2\}) = \top$, allora $c \subseteq \gamma(+)$ sarebbe vero ma $\alpha(c) = \top \not\sqsubseteq_A +$, e la 2 fallirebbe: α non sarebbe la migliore astrazione.

11 Lezione 11 del 15 ottobre 2026

Riprendiamo dagli argomenti della lezione 10 e proviamo a dimostrare per la constant propagation le seguenti proprietà:

- α e γ sono monotone
- $\alpha \circ \gamma$ è estensiva, $\forall c \in C.c \sqsubseteq_C \gamma(\alpha(c))$
- $\gamma \circ \alpha$ è riduttiva, $\forall a \in A.\alpha(\gamma(a)) \sqsubseteq_A a$

Esempio 28

I reticoli presi in considerazione sono quelli dell'esempio della scorsa lezione, Figura 9 e Figura 10.

Partiamo col dimostrare se α e γ sono monotone:

1. α è monotona, $\forall X, Y \in \mathcal{P}(\mathbb{Z}), X \subseteq Y \Rightarrow \alpha(X) \subseteq \alpha(Y)$
   - $X = \varnothing$, per def. di α, $\alpha(\varnothing) = \bot$, per def. di $\sqsubseteq_{\text{CP}}$ $\alpha(X) = \bot \sqsubseteq_{\text{CP}} \alpha(Y)$, per qualunque Y.
   - $|X| = 1, X = \{v\}$, per def. di α, $\alpha(\{v\}) = v$
     - $X = Y = \{v\}$, banale.
     - $X \subset Y$, per def. di $\subseteq_{\text{CP}}$ $|Y| > |X| = 1$, per def. di α, $\alpha(Y) = \top$; quindi $\alpha(X) = v \sqsubseteq_{\text{CP}} \alpha(Y) = T$
   - $|X| > 1, X \subseteq Y \Rightarrow |Y| > 1$, per def. di $\sqsubseteq_{\text{CP}}$: $\alpha(X) = \top \sqsubseteq_{\text{CP}} \alpha(Y) = \top$
2. γ è monotona, $\forall x, y \in \text{CP}$, se $x \sqsubseteq_{\text{CP}} y \Rightarrow \gamma(x) \sqsubseteq_{\text{CP}} \gamma(y)$
   - $x = \bot$, per def. di $\gamma(\bot) = \varnothing$, e per def. di $\subseteq$, $\gamma(x) = \varnothing \subseteq \gamma(y)$, per qualunque $y \in \text{CP}$.
   - $y = \top$, per def. di $\gamma(T) = \mathbb{Z}$, per def. di $\subseteq$, $\gamma(x) \subset \mathbb{Z} = \gamma(y)$, per qualunque $y \in \text{CP}$.
   - $x = y = v$, per def. di γ, $\gamma(x) = \gamma(y) = \{v\}$
3. $\gamma \circ \alpha$ è estensiva: $\forall I \in \mathcal{P}(\mathbb{Z}).I \subseteq \gamma(\alpha(I))$
   - $I = \varnothing$, per def. di α, $\alpha(I) = \bot$, def. γ, $\gamma(\bot) = \varnothing \Rightarrow \varnothing \subseteq \gamma(\alpha(\varnothing))$.
   - $I = \{v\}$, def. α, $\alpha(\{v\}) = v$, def. γ, $\gamma(v) = v \Rightarrow \{v\} \subseteq \gamma(\alpha(\{v\}))$.
   - $|I| > 1$, def. α, $\alpha(I) = \top$, def γ, $\gamma(T) = \mathbb{Z} \Rightarrow I \subseteq \gamma(\alpha(I))$.

Nota

Per indicare le connessioni di Galois useremo la seguente notazione

$$C \underset{\gamma}{\overset{\alpha}{\leftrightarrows}} A$$

11.1 Inserzioni di Galois

Dalle connessioni di Galois possiamo derivare le inserzioni di Galois.

Diamo una definizione formale di inserzione di Galois.

Definizione 39 — Inserzione di Galois

Presi due poset $< C, \sqsubseteq_C >$ e $< A, \sqsubseteq_A >$ con $\alpha : C \to A$ e $\gamma : A \to C$, supponendo di avere una connessione di Galois tra C ed A tramite α e γ, se $\alpha \circ \gamma$ è l'identità allora è chiamata **inserzione di Galois**.

In questo caso l'astrazione è suriettiva, mentre la croncretizzazione è iniettiva.

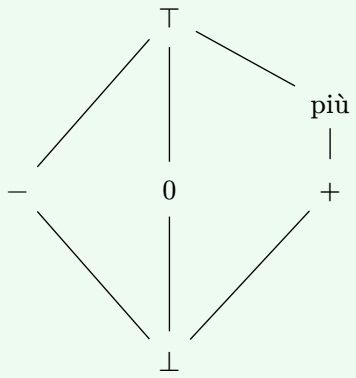
> 🔁 **Richiamo**
>
> **Suriettiva**: Ogni elemento del codominio è associato ad almeno un elemento del dominio.
> **Iniettiva**: Ogni elemento del codominio ha al più un elemento del dominio.

Ogni connessione di Galois può essere ridotta ad una inserzione di Galois.

Quando non abbiamo una inserzione di Galois, abbiamo almeno due elementi astratti che rappresentano la stessa informazione, di conseguenza elementi ridondanti, possiamo rimuoverli con una riduzione del dominio astratto.

Esempio 29

Se abbiamo elementi ridondanti nella nostra astrazione, possiamo rimuoverli, per esempio prendiamo in considerazione il seguente reticolo per l'analisi dei segni:



In questo caso «più» è «+» rappresentano la stessa informazione, quindi possiamo ridurre tutto ad utilizzare un solo simbolo dei due. Così facendo otteniamo un'inserzione di Galois.

Per l'inserzione di Galois vogliamo che l'identità sia definita da $\alpha \circ \gamma$, ma nel caso in cui dovessimo avere l'identità definita da $\gamma \circ \alpha$?

Vuol dire che abbiamo creato un isomorfismo, di conseguenza i problemi di decidibilità del contesto concreto si spostano nella nostra astrazione. Significa anche che non abbiamo perso informazioni dal passaggio dal concreto all'astratto, noi vogliamo perdere informazioni per poter guadagnare in decidibilità.

Se riprendiamo la definizione alternativa di connessione di Galois Definizione 38, vogliamo sempre rispettare la condizione di **soundness**, mentre se possibile ottenere la **best abstraction**.

È anche possibile che non esista una connessione di Galois tra C ed A, dovuta alla non esistenza della miglior astrazione, per esempio un poliedro convesso.

Vediamo un esempio sulla best abstraction:

Esempio 30

Consideriamo di voler approssimare una circonferenza: $P = \{(x, y) \mid x^2 + y^2 \leq 1\}$

Possiamo usare un poliedro convesso: $a * x + b * y \leq c$

Possiamo trovare molte sovra-approssimazioni per la circonferenza.

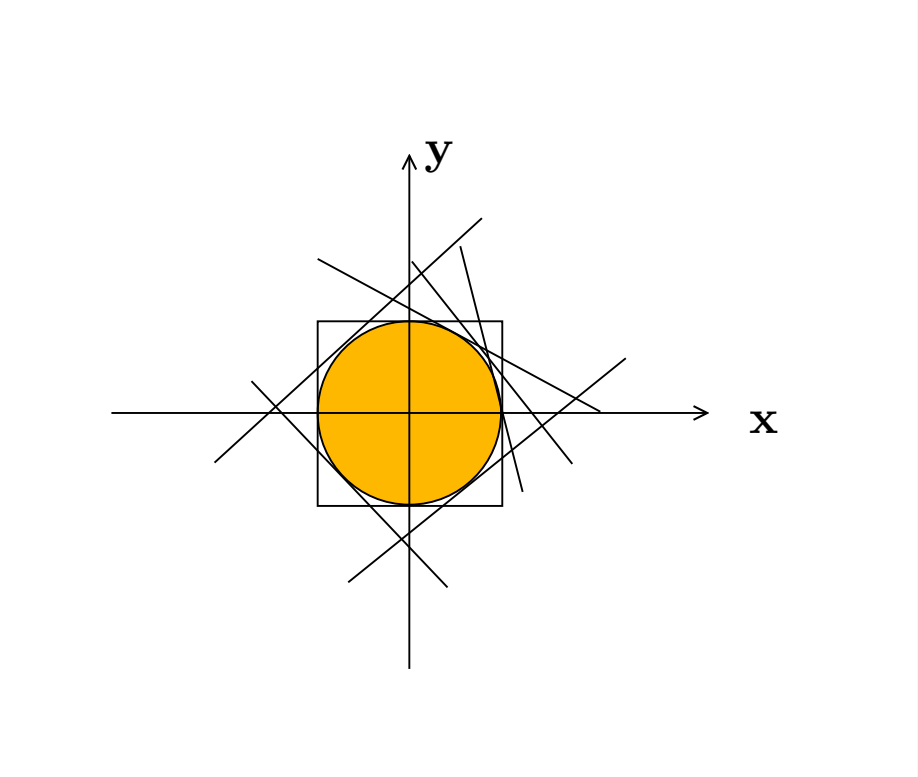


Figura 13. Approssimazione circonferenza con poliedro convesso.

Un altro esempio fatto a lezione è quello sul trovare la minima astrazione per i DFA.

Esempio 31

Obiettivo. Vogliamo mostrare che sul reticolo dei DFA **non esiste una best abstraction** α: cioè non possiamo definire una funzione di astrazione α tale che (α, γ) sia una connessione di Galois.

11.1.1 Passo 0: i due domini

- **Dominio concreto.** Fissiamo un alfabeto, ad esempio $\Sigma = \{a, b\}$. Gli elementi concreti sono i **linguaggi**, cioè insiemi di stringhe, quindi il dominio è

$$\langle \mathcal{P}(\Sigma^*), \subseteq \rangle$$

Un linguaggio qualsiasi è permesso, anche uno molto complicato.

- **Dominio astratto.** Sono i DFA, dove due automi che riconoscono lo stesso linguaggio sono considerati lo stesso elemento:

$$\langle \mathrm{DFA}/(\equiv), \sqsubseteq_{\mathrm{DFA}}, \sqcap_{\mathrm{DFA}}, \sqcup_{\mathrm{DFA}}, \top_{\mathrm{DFA}}, \bot_{\mathrm{DFA}} \rangle$$

  con $A \equiv B$ se $L(A) = L(B)$ e $A \sqsubseteq_{\mathrm{DFA}} B$ se $L(A) \subseteq L(B)$. In pratica gli elementi astratti sono i **linguaggi regolari** (quelli riconosciuti da un DFA).

> **Nota**
>
> Prendiamo in considerazione le classi di equivalenza dei DFA con $\mathrm{DFA}/(\equiv)$, in modo da considerare un solo DFA per automi che riconoscono lo stesso linguaggio. Quindi per $L(A) \equiv L(B)$ avranno lo stesso DFA che li rappresenta.

- **Concretizzazione.** È la funzione più naturale: dato un DFA, restituisci il linguaggio che riconosce

$$\gamma_{\mathrm{DFA}}(A) = L(A)$$

11.1.2 Passo 1: cosa dovrebbe fare α

Una connessione di Galois richiede, per ogni linguaggio L e ogni DFA R:

$$\alpha(L) \sqsubseteq_{\mathrm{DFA}} R \iff L \subseteq \gamma_{\mathrm{DFA}}(R)$$

Leggiamola in italiano: «$\alpha(L)$ è più piccolo di R se e solo se R contiene tutte le stringhe di L». Ne seguono due fatti:

1. Prendendo $R = \alpha(L)$ nella formula otteniamo $L \subseteq \gamma_{\mathrm{DFA}}(\alpha(L))$. Quindi $\alpha(L)$ è una **sovra-approssimazione** di L: riconosce tutte le stringhe di L (ed eventualmente altre).
2. Per ogni altra sovra-approssimazione R di L vale $\alpha(L) \sqsubseteq_{\mathrm{DFA}} R$. Quindi $\alpha(L)$ è la **più piccola** tra tutte le sovra-approssimazioni regolari di L.

Riassumendo: $\alpha(L)$, se esiste, deve essere il **minimo** tra tutti i linguaggi regolari che contengono L. Questa è proprio la best abstraction. Quindi: per far fallire α ci basta trovare un L per cui questo minimo **non esiste**.

11.1.3 Passo 2: scegliamo il linguaggio «cattivo»

Prendiamo

$$L = \{a^n b^n : n \geq 0\} = \{\varepsilon, ab, aabb, aaabbb, ...\}$$

cioè le stringhe formate da un po' di a seguite da **esattamente lo stesso numero** di b.

Questo linguaggio **non è regolare**: per riconoscerlo un automa dovrebbe contare quante a ha letto, ma un DFA ha un numero finito di stati e quindi non può contare fino a un numero

arbitrario (lo si dimostra col pumping lemma). Quindi nessun DFA riconosce esattamente L. In compenso L è un elemento perfettamente valido del dominio concreto $\mathcal{P}(\Sigma^*)$.

11.1.4 Passo 3: dimostrazione per assurdo

Supponiamo per assurdo che $\alpha(L)$ esista. Per il Passo 1 sarebbe un linguaggio regolare R con due proprietà: R contiene L e R è il più piccolo linguaggio regolare che contiene L.

1. **$R \neq L$.** Sappiamo che R è regolare e che L non lo è, quindi non possono essere lo stesso insieme. Siccome $L \subseteq R$, ne segue che R ha **strettamente più stringhe** di L.
2. **Esiste una stringa «in più».** Per il punto precedente c'è almeno una stringa w con $w \in R$ e $w \notin L$.
3. **Togliamo w.** Definiamo $R' = R \setminus \{w\}$, cioè R senza quella sola stringa.
4. **R' è ancora regolare.** Togliere una stringa (o un numero finito di stringhe) a un linguaggio regolare lo lascia regolare. Infatti $R' = R \cap \overline{\{w\}}$, e i linguaggi regolari sono chiusi per intersezione e complemento (basta modificare l'automa di R in modo che rifiuti w).
5. **R' contiene ancora L.** Abbiamo tolto solo w, e $w \notin L$, quindi tutte le stringhe di L sono ancora in R'.
6. **R' è strettamente più piccolo di R:** $R' \subset R$.

Allora R' è un linguaggio regolare che contiene L ed è **più piccolo** di R. Questo contraddice il fatto che R fosse il più piccolo. L'assurdo nasce dall'aver supposto che $\alpha(L)$ esistesse. □

> **Nota**
>
> Il ragionamento in una frase: **qualunque sovra-approssimazione regolare di L possiamo scegliere, ce n'è sempre una più precisa**, perché basta togliere una stringa che non sta in L. Quindi non c'è mai una «migliore».

11.1.5 Passo 4: l'intuizione con una catena infinita

Possiamo vederlo anche in modo concreto. Per ogni $k \geq 0$ sia

$$R_k = \{a^n b^n : n \leq k\} \cup \{a^n b^m : n > k \ \text{ oppure } \ m > k\}$$

Cioè: teniamo esattamente le stringhe di L con $n \leq k$, e poi accettiamo «alla rinfusa» tutte le stringhe del tipo $a^n b^m$ con almeno uno dei due esponenti oltre k. Ogni R_k è regolare (la prima parte è finita, la seconda richiede solo di contare fino a k) e contiene L. Inoltre diventano sempre più precisi:

$$R_0 \supseteq R_1 \supseteq R_2 \supseteq \ldots$$

e la loro intersezione è proprio L, che non è regolare. Abbiamo quindi **approssimazioni sempre migliori che non arrivano mai a una migliore**: ci si avvicina a L senza poterlo raggiungere.

11.1.6 Conclusione

- L'insieme dei linguaggi regolari è un reticolo (unioni e intersezioni **finite** di regolari sono regolari), ma **non è un reticolo completo**: l'intersezione di infiniti regolari può non essere regolare, come nella catena sopra.
- Per questo la funzione α **non è definibile** su tutto $\mathcal{P}(\Sigma^*)$ e non abbiamo una connessione di Galois tra $\mathcal{P}(\Sigma^*)$ e i DFA.
- Si può comunque usare questo dominio rinunciando alla best abstraction: si richiede solo la **soundness** (ogni astrazione scelta è una sovra-approssimazione corretta, cioè $L \subseteq \gamma_{\text{DFA}}(A)$). È il punto di vista della definizione alternativa di connessione di Galois vista a lezione.

Anche se non abbiamo una connessione di Galois tra concreto e astratto, possiamo comunque fare interpretazione astratta.

11.2 Semantica concreta

Nelle prime lezioni abbiamo introdotto la semantica concreta e la collecting semantics per il linguaggio IMP Sezione 5.1 e Sezione 7.1. La collecting semantics ci torna utile quando lavoriamo direttamente su CFG, noi faremo interpretazione astratta basandoci sui CFG di un programma.

Prendiamo in considerazione il seguente codice e il corrispettivo CFG.

Esempio 32

```
1       x = 0;
2  while  (x < 100)
3     if  (x < 50)
4       x = x + 2;
5         else
6       x = x + 10;
```

Codice 1. Esempio codice.

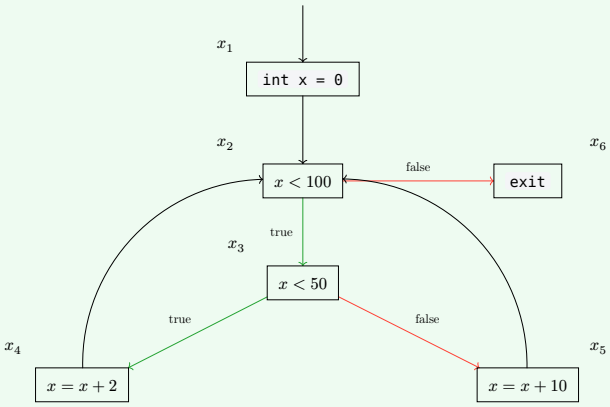


Figura 14. Control Flow Graph.

Il nostro obiettivo è calcolare gli stati entranti e uscenti per ogni program point x_i.

- **Stato entrante**: è l'upper bound degli stati in uscita dai blocchi che puntano a x_i. Il blocco x_1 sarà lo stato iniziale.
- **Stato uscente**: è l'applicazione della collecting semantics dello statement applicata allo stato entrante.

Passiamo a calcolare gli stati entranti per ogni program point x_i. Scegliamo un program point e guardiamo tutti gli stati uscenti che puntano a quel program point.

- $x_1 = I$.
- $x_2 = [\![x = 0]\!]x_1 \cup [\![x = x + 2]\!]x_4 \cup [\![x = x + 10]\!]x_5$

- $x_3 = [\![x < 100]\!]x_2$, questi sono i valori filtrati per $x < 100$.
- $x_4 = [\![x < 50]\!]x_3$, valori filtrati per $x < 50$.
- $x_5 = [\![x \geq 50]\!]x_3$, valori filtrati per $x \geq 50$.
- $x_6 = [\![x \geq 100]\!]x_2$, valori filtrati per $x \geq 100$.

Dati m nodi di un CFG, vorremmo risolvere il seguente sistema d'equazioni:

$$F = \left\{ x_i = f_{i(x_1,...,x_m)} \mid i = 1, 2, ..., m \right\}$$

Possiamo calcolare la soluzione minima per F come il limite delle iterazioni di **Kleene** Teorema 3 sul poset:

$$< \mathcal{P}(\mathbb{Z}), \subseteq, \cap, \cup, \mathbb{Z}, \varnothing >$$

Ovvero calcoliamo il least fix point di F su x:

$$x^{\text{lfp}(F)} = \left(x_1^{\text{lfp}(F)}, ..., x_m^{\text{lfp}(F)} \right)$$

Il problema è che il calcolo di questo fix point è **indecidibile**, il fix point esiste Teorema 2, ma in generale non è calcolabile.

12 Lezione 12 del 16 ottobre 2026

Verso la fine della lezione precedente abbiamo fatto un esempio sul calcolo del punto fisso su un programma rappresentato da un CFG. Abbiamo stabilito che il punto fisso esiste, ma non è calcolabile.

12.1 Semantica astratta

Possiamo calcolare il punto fisso passando dal contesto concreto ad un contesto astratto, questo ci porta una perdita di informazione ma guadagnamo in decidibilità.

Esempio 33

Riprendiamo l'esempio Esempio 32 della scorsa lezione.

Vogliamo calcolare il punto fisso del sistema di equazioni per F, dato che nel concreto non è calcolabile passiamo ad una astrazione per poter calcolare il punto fisso:

Riprendiamo il sistema di equazioni nel contesto concreto:

$$F = \begin{cases} x_1 = I \in \mathcal{P}(\mathbb{Z}) \\ x_2 = [\![x = 0]\!]x_1 \cup [\![x = x + 2]\!]x_1 \cup [\![x = x + 10]\!]x_5 \\ x_3 = [\![x < 100]\!]x_2 \\ x_4 = [\![x < 50]\!]x_3 \\ x_5 = [\![x \geq 50]\!]x_3 \\ x_6 = [\![x \geq 100]\!]x_2 \end{cases}$$

Il nostro obiettivo sarebbe calcolare $\mathrm{lfp}^{\subseteq}_{\varnothing}$, il fix point esiste perché è un reticolo completo, ma nel concreto non possiamo calcolarlo.

Per il teorema di Kleene Teorema 3, possiamo trovare un punto fisso dopo un determinato numero di iterazioni sulla funzione F.

Partiamo calcolando F_0:

$$F_0 = \begin{cases} x_1 = \varnothing \\ x_2 = \varnothing \\ x_3 = \varnothing \\ x_4 = \varnothing \\ x_5 = \varnothing \\ x_6 = \varnothing \end{cases}$$

Tutte le equazioni vengono inizializzate a 0, ora calcoliamo l'iterazione 1 per la funzione F, per il passo successivo prendiamo la definizione di F, e applichiamo ad ogni x_i la sua equazione con la valutazione della semantica in base ai valori entranti nei blocchi del CFG e otteniamo:

$$F_1 = \begin{cases} x_1 = \mathbb{Z} \\ x_2 = \{0\} \\ x_3 = \{0\} \\ x_4 = \{0\} \\ x_5 = \varnothing \\ x_6 = \varnothing \end{cases}$$

Calcoliamo F_2 prendendendo i valori di F_1 e calcoliamo quelli nuovi; ci sarebbero molte più iterazioni da fare per arrivare al fixpoint ma indichiamo con F_m il fixpoint della funzione F.

$$F_2 = \begin{cases} x_1 = \mathbb{Z} \\ x_2 = \{0, 2\} \\ x_3 = \{0, 2\} \\ x_4 = \{0, 5\} \\ x_5 = \varnothing \\ x_6 = \varnothing \end{cases} \qquad F_m = \begin{cases} x_1 = \mathbb{Z} \\ x_2 = \{0, 2, 4, ...50, 60, 70, ..., 100\} \\ x_3 = x_2 \setminus \{100\} \\ x_4 = \{0, 2, ..., 48\} \\ x_5 = \{50, ..., 90\} \\ x_6 = \{100\} \end{cases}$$

In questo caso è calcolabile perché il programma è costante, ma in generale il fix point nel concreto è non decidibile.

Siccome nel contesto concreto, in generale, non è possibile calcolare il least fixpoint sul dominio C, l'idea è quella di passare al calcolo di un fixpoint astratto su un dominio astratto A che approssima correttamente quello concreto.

Quello che vogliamo fare è:

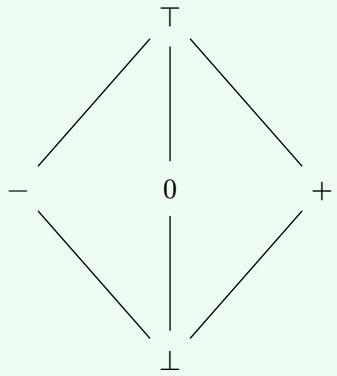
- Calcolare $\alpha(\text{lfp}(F))$, F concreta.
- Senza effettivamente calcolare $\text{lfp}(F)$.

Il primo passo è quello di spostare la computazione dal dominio C al dominio A.

Facciamo un esempio di analisi dei segni definendo una semantica astratta per il linguaggio Imp Sezione 5.1.

Esempio 34

Riprendiamo il reticolo dei segni:



Riprendiamo anche le funzioni α e γ:

$$\alpha : \mathcal{P}(\mathbb{Z}) \to \text{Sign}$$

$$\gamma : \text{Sign} \to \mathcal{P}(\mathbb{Z})$$

Dobbiamo definire la funzione della memoria astratta per la nostra semantica astratta:

$$M^{\#} : \mathbb{V} \to \text{Sign}$$

Abbiamo la funzione $M^{\#}$, che prende in input una variabile $v \in \mathbb{V}$, $\mathbb{V}$ è l'insieme delle variabili, e restituisce un Sign. Passiamo ora a definire la semantica astratta:

- $(\!|\cdot|\!)^{\#} : M^{\#} \to \text{Sign}$; questa è la semantica astratta per le espressioni.
- $[\![\cdot]\!]^{\#} : M^{\#} \to M^{\#}$; questa è la semantica astratta per gli statement.
- $[\![x = e]\!]^{\#}\ m^{\#} \to m^{\#}[(\!|e|\!)^{\#} m^{\#}]$
- $[\![\text{skip}]\!]^{\#}\ m^{\#} = m^{\#}$
- $(\!|e_1 + e_2|\!)^{\#}\ m^{\#} = (\!|e_1|\!)^{\#}\ m^{\#} +^{\#} (\!|e_2|\!)^{\#}\ m^{\#}$

Notare che nell'ultima regola abbiamo l'operatore $+^{\#}$, questo è l'operatore delle addizioni per la nostra semantica astratta, possiamo definire la tabella con tutti i possibili risultati per l'operatore $+^{\#}$.

$+^{\#}$	$\top$	$+$	$-$	0	$\bot$
$\top$	$\top$	$\top$	$\top$	$\top$	$\bot$
$+$	$\top$	$+$	$\top$	$+$	$\bot$
$-$	$\top$	$\top$	$-$	$-$	$\bot$
0	$\top$	$+$	$-$	0	$\bot$
$\bot$	$\bot$	$\bot$	$\bot$	$\bot$	$\bot$

Ci sarebbero da definire anche gli altri operatori astratti per sottrazione, divisione e moltiplicazione. Tutti quanti seguono la stessa struttura dell'operatore $+^{\#}$.

Dobbiamo valutare anche le espressioni booleane per la nostra semantica astratta, qui le cose si fanno un po' più articolate rispetto agli operatori aritmetici. Supponiamo di avere la seguente memoria con una guardia:

$$[x \to \top, y \to +]$$

$$x \geq 100;$$

Guardando la memoria in ingresso alla guardia, cosa possiamo dire su x?

Sicuramente x una volta superata la guardia avrà per forza valori positivi, dato che x deve essere maggiore o uguale a 100, quindi da $\top$ possiamo raffinare a $+$, di conseguenza in uscita dalla guardia la nuova memoria sarà:

$$[x \to +, y \to +]$$

Prendiamo in considerazione un altro caso:

$$[x \to \top, y \to +]$$

$$y < -100;$$

Qui invece cosa possiamo dire su y?

In ingresso alla guardia ci aspettiamo che y assuma valori positivi, mentre la guardia ci chiede che y minore di -100 e quindi negativa. In questo caso in memoria avremo $\bot$ per la variabile y. Dunque la nuova memoria sarà:

$$[x \to \top, y \to \bot]$$

Guardiamo un ultimo esempio, prendiamo in considerazione la memoria e la guardia:

$$[x \to +, y \to -]$$

$$x + y \leq z^2$$

Cosa possiamo dire su x, y e z?

In questo caso, quando abbiamo più variabili in relazione con l'operatore di confronto non possiamo raffinare la nostra analisi dei segni, di conseguenza dobbiamo propagare la stessa memoria dopo la guardia.

Invece per questo esempio:

$$[x \to \top]$$

$$x < -;$$

Qui cosa possiamo dire su x?

Sicuramente avrà un valore negativo, quindi per nella nuova memoria avremo $[x \to -]$ per ottenere questa valutazione dobbiamo fare il **glb** tra $\top$ e il segno meno. Stessa cosa per il caso :

$$[x \to \top]$$

$$0 < x;$$

In questo caso si fa il **glb** tra $\top$ e $+$ e nella nuova memoria avremo x con segno positivo.

Da questi esempi abbiamo visto che possiamo dedurre i valori di x o y in base all'operatore di confronto e al segno dell'altra espressione usata nel confronto, dobbiamo definire gli operatori astratti per le espressioni booleane della nostra semantica astratta.

Iniziamo definendo l'operatore per le espressioni booleane:

- $assume : \text{Bexp} \times M^\# \to M^\#$

Il modo più semplice per definire l'operatore booleano $<$ è quello di farlo per casi:

- $assume_{e_1<e_2}(m^\#) = \begin{cases} m^\#[x \leftarrow m^{\#(x)} \sqcap -] & \text{se } e_1{=}x \in \mathbb{V} \wedge (e_2)^\# \; m^\# \in \{0,-\} \\ m^\#[x \leftarrow m^\#(x) \sqcap +] & \text{se } e_2{=}x \in \mathbb{V} \vee (e_1)^\# \; m^\# \in \{0,+\} \\ \bot_{M^\#} & \text{se } (e_1)^\# \; m^\# {=} \bot \vee (e_2)^\# m^\# {=} \bot \\ m^\# & \text{altrimenti} \end{cases}$

Possiamo definire anche gli altri operatori booleani per la semantica astratta:

- $assume_{\text{true}}(m^\#) = m^\#$
- $assume_{\text{false}}(m^\#) = \bot_{M^\#}$
- $assume_{e_1>e_2 \; / \; =<,\geq,==,\; !=}$ → questi operatori sono simili a $assume_{e_1<e_2}$.
- $assume_{e_1 \;\&\&\; e_2}(m^\#) = assume_{e_1}(m^\#) \sqcap assume_{e_2}(m^\#)$
- $assume_{e_1 \;||\; e_2}(m^\#) = assume_{e_1}(m^\#) \sqcup assume_{e_2}(m^\#)$

Con questo esempio abbiamo spostato il calcolo della decidibilità dal mondo concreto all'astratto, ma dobbiamo assicurarci che la nostra astrazione sia sound, ovvero che la nostra astrazione copra ogni comportamento reale del programma. Ritorniamo brevemente all'operatore astratto $+^\#$.

$$+ \quad +^\# \quad - = \top \quad \text{questo è sound.}$$

$$+ \quad +^\# \quad - = - \quad \text{questo NON è sound.}$$

12.2 Soundness di un'astrazione

Supponiamo che C e A formino una connessione di Galois tramite α e γ e sia $f : C \to C$ un'operazione concreta e $f^{\#} : A \to A$ un'astrazione dell'operazione concreta f. $f^{\#}$ è sound se:

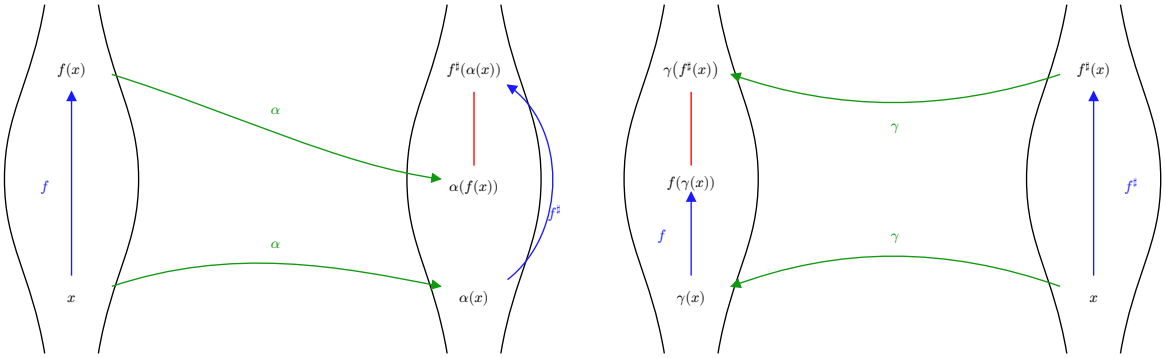
$$\forall c \in C.\alpha(f(x)) \sqsubseteq_A f^{\#}(\alpha(x))$$

oppure vale anche:

$$\forall a \in A.f(\gamma(a)) \sqsubseteq_C \gamma\big(f^{\#}(a)\big)$$

Graficamente possiamo vederla cosi:

Backward **Forward**



(a)

$$\forall c \in C.\alpha(f(c)) \sqsubseteq_A f^{\sharp}(\alpha(c))$$

(b)

$$\forall a \in A.f(\gamma(a)) \sqsubseteq_C \gamma\big(f^{\sharp}(a)\big)$$

12.2.1 Best correct approximation

Per un dominio C possiamo avere molteplici possibili astrazioni, quindi dobbiamo chiederci quale sia la l'astrazione $f^{\#}$ più precisa tra tutte le possibili astrazioni per f?

Supponiamo che C ed A formino una connessione di Galois tramite α e γ e sia $f : C \to C$ un'operazione concreta. La migliore aprrossimazione corretta (best correct approximation) $f^{\text{bca}} : A \to A$ è definita come:

$$f^{\text{bca}} = \alpha \circ f \circ \gamma$$

La $f^{\text{bca}} = \alpha(f(\gamma(a)))$ è spesso indecidibile, dovremmo concretizzare a ottenendo $\gamma(a)$, che di solito è un insieme infinito, per i segni avremmo $\gamma(+) = \{1, 2, 3, ...\}$ e poi applicare la f concreta a tutti gli elementi e astrarre il risultato di f con α.